

A consistency proof for BGA carried out inside grounded arithmetic in Isabelle/Pure

Omar Muhammad

August 19, 2026

Abstract

Grounded arithmetic admits arbitrary recursive definitions by weakening inference rules with *habeas quid* termination premises. The existing mechanized consistency results for grounded arithmetic are carried out in Isabelle/HOL, that is, in a classical metatheory far stronger than the object logic. This report documents a development that carries the argument out inside a version of grounded arithmetic itself. QGA, an axiomatization of grounded arithmetic with quantifiers, is embedded into Isabelle/Pure in `GD.thy`. Basic grounded arithmetic BGA is arithmetized on top of it in `BGA_on_GA.thy`: terms, formulas, judgments, proofs, the operational semantics and the proof checker are encoded as QGA natural numbers, and the checker is proved sound with respect to the encoded semantics under a small set of abstract assumptions about the encoding. A concrete Cantor-pairing encoding in `encode.thy` discharges those assumptions and yields, as a theorem of QGA: for any two numbers coding BGA terms a and b , no numbers p_1 and p_2 code BGA proofs, from empty hypotheses, of $a = b$ and of $a \neq b$ respectively.

Contents

1	Overview	3
1.1	The three layers	3
1.2	What is proved	3
1.3	The two levels	3
2	QGA	4
2.1	Formulas and terms	5
2.2	Structural rules	5
2.3	Propositional rules	5
2.4	Equality and the <i>habeas quid</i> judgments	6
2.5	Natural numbers	7
2.6	Quantifiers	8
2.7	Conditionals	9
2.8	The definitional mechanism	10
2.9	Arithmetic	10
2.10	Induction principles	11
2.11	Pairing	11
2.12	Lists	12
2.13	Summary	13

3	BGA arithmetized in QGA	13
3.1	The system being encoded	13
3.2	Codes	16
3.3	The locale tower	16
3.4	The consistency skeleton	17
3.5	Encoded syntax	19
3.6	bga_encoding	19
3.7	bga_dfns	21
3.8	Semantics	22
3.9	Satisfaction at a fixed budget	23
3.10	bga_subst and asn_put	24
3.11	The proof rules of BGA	25
3.12	bga_subst_rule	26
3.13	bga_eq_rule	28
3.14	bga_ind_rule	29
3.15	bga_struct_rule	30
3.16	bga_app_rule	30
3.17	bga_proof_check	31
4	The consistency proof	33
4.1	Shape of the argument	33
4.2	The evaluator is a partial function	33
4.3	Soundness of a single step	34
4.4	From steps to proofs	36
4.5	Consistency	37
5	The concrete encoding	38
5.1	The encoders	38
5.2	Concrete definitions	39
5.3	Discharging the assumptions	40
5.4	Instantiation and the theorem	42
6	Reduction of QGA to BGA	43
7	A big-step semantics for QGA	43
7.1	What the proof needs	43
7.2	Terms	43
7.3	Formulas	43
7.4	The quantifier clause	44
7.5	Obligations	45
7.6	What the semantics does not settle	45
8	QGA on QGA: feasibility and the development skeleton	46
8.1	Is it possible?	46
8.2	Where the plan of §7 does not close	46
8.3	The two changes that make it close	47
8.4	The clause that removes the completeness obligation	47
8.5	What the development contains	48
8.6	Status and what remains	49
8.7	What this would and would not settle	50
9	Status	50

1 Overview

1.1 The three layers

The development consists of three theories, each importing the previous one.

Theory	Imports	Contributes
<code>GD.thy</code>	Pure	QGA, an embedding of grounded arithmetic in Isabelle/Pure
<code>BGA_on_GA.thy</code>	GD	BGA arithmetized in QGA, and the soundness proof for its proof checker
<code>encode.thy</code>	BGA_on_GA	a concrete encoding discharging every assumption, and the consistency theorem

`BGA_on_GA.thy` does the heavy lifting. It never commits to a particular Gödel numbering. It fixes an interface (`tag_T`, `load_T`, `pack_T` and their formula counterparts) together with the smallest set of assumptions under which the soundness argument goes through, and proves everything relative to that interface. `encode.thy` then pays the arithmetic cost once, by exhibiting one encoding that satisfies the assumptions, and instantiates the result.

1.2 What is proved

The final theorem, `BGA_syntactically_consistent`, says that for any two numbers coding BGA terms a and b , no numbers p_1 and p_2 code BGA proofs, from empty hypotheses, of $a = b$ and of $a \neq b$ respectively.

QGA is stronger than BGA, so this is a consistency proof of a weaker system inside a stronger one, in the ordinary Gödelian arrangement. Its interest lies elsewhere. It shows that grounded arithmetic can carry a full metamathematical argument about a formal system with no classical ambient logic available at any point. Turning it into a self-consistency result requires a reduction of QGA back to BGA, which is the subject of §6 and is left for later work.

1.3 The two levels

Two logics are in play throughout, and separating them is worth doing before any rules appear.

QGA is the logic under study and that we will use for proof writing. Its formulas are the terms of type `o`. Its vocabulary, given in full in §2, consists of the primitives $\vee$ and $\neg$, the connectives $\wedge$, $\rightarrow$ and $\leftrightarrow$ defined from them, the equality $=$, and the quantifiers $\forall$ and $\exists$.

Isabelle/Pure is the framework in which QGA is written down. Pure is a small calculus whose purpose is to state the rules of other logics, and it has three symbols. $A \Rightarrow B$ says that from a derivation of A one obtains a derivation of B , so a rule “from A infer B ” is written this way. $\wedge x. A$ says that A was derived for an arbitrary x , which is how the eigenvariable condition of a rule is expressed: the premise $\wedge x. x \in N \Rightarrow Q \ x$ of `forallI` records that Q holds of a variable about which nothing was assumed. $a \equiv b$ is the definitional equality of the framework, used where a QGA notion is abbreviated rather than axiomatized.¹

The type system keeps the two levels apart. `o` is the type of QGA formulas and `prop` the type of Pure judgments, connected by `Trueprop :: o  $\Rightarrow$  prop`, which is declared with the `judgment` command and printed invisibly. Nothing maps `prop` back to `o`. A QGA formula can therefore be asserted as a judgment, while $\Rightarrow$ and $\wedge$ can never appear inside a QGA formula.

¹Definitional equality identifies terms that differ only in ways the logic cannot see: renaming a bound variable (α), applying a lambda to its argument (β), and replacing $\lambda x. f \ x$ by $f \ (\eta)$. Unfolding a `definition` is also an $\equiv$ step.

Pure holds hypothetical judgments freely, because $\Rightarrow$ may always be introduced. Converting one into a QGA formula requires `implI`, whose premise $a \vdash B$ demands that the antecedent be decided. *Habeas quid* is a gate between the two levels, and every implication occurring in a QGA formula has passed through it.

This arrangement is what supports the claim that the development is carried out in QGA rather than in Isabelle. Pure’s kernel offers assumption, introduction and elimination for $\Rightarrow$ and $\wedge$, the rules for $\equiv$, instantiation of schematic variables, and citation of an axiom. Only the last of these introduces an assertion. The others move an assertion between hypothesis and antecedent position, replace a rule schema by one of its instances, or work on equalities. Remove the axioms of `GD.thy` and Pure proves no assertion at all except from an assumption of that same assertion, since to Pure the QGA connectives are constants about which it knows nothing. Every QGA formula established in the development therefore comes from a QGA rule. **[Omar: This is my hand wave for the claim. I want to do rigorous analysis + a consistency proof of QGA in HOL soon.]**

One kernel rule needs care. From $A \equiv B$ and A one may infer B , so an equality can introduce an assertion that appears nowhere in its premises. Inside Pure, equalities come only from the conversions described in the footnote above, none of which changes meaning. They can also come from an axiom, and two axioms of `GD.thy` are of that kind: `eq_reflection` ($x = y \Rightarrow x \equiv y$) and `iff_reflection` ($p \leftrightarrow q \Rightarrow p \equiv q$), which exist so that Isabelle’s simplifier can act as a derived rule. These two are the entire surface through which the framework could contribute strength. Each mirrors a QGA rule together with congruence, `eq_reflection` mirroring `eqSubst`, whose context $Q :: \text{num} \Rightarrow o$ is already arbitrary, so both are expected to be pretty tame. **[Omar: I want to do a rigorous analysis on this too.]**

A last reliance on the framework concerns the quantifier syntax. QGA formulas are the terms of type o , and `forall` has type $(\text{num} \Rightarrow o) \Rightarrow o$, so a quantified formula is `forall` applied to a Pure function. Binding is therefore the framework’s, and renaming, capture-avoiding substitution and freshness come for free rather than being axiomatized. This is the usual higher-order abstract syntax treatment of a binder and is what keeps §2 free of a substitution calculus. **[Omar: I think the minor concern here would be that the bodies one may quantify over are the Pure terms of type $\text{num} \Rightarrow o$ over the QGA signature, so the formulas of QGA are not given by an inductive definition. So an arithmetization of QGA must represent quantifier bodies by a first-order syntax with explicit variables.]**

In the natural-deduction presentations below, QGA rules are written with premises above the line and the conclusion below, with no explicit context. A premise discharged by a rule is written in brackets above the derivation that uses it. BGA judgments carry an explicit list of hypotheses, so BGA rules are written as sequents $\Gamma \vdash \varphi$. All Isabelle snippets are extracted verbatim from the theories by `extract.sh`, which replaces Isabelle’s ASCII escapes by the corresponding Unicode glyphs and changes nothing else.

2 QGA

`GD.thy` axiomatizes grounded arithmetic as an object logic over Isabelle/Pure. The propositional and arithmetic core, and the definitional mechanism are from Kehrli’s development, who established that grounded arithmetic can be axiomatized in Pure and reasoned in productively.²

²Sascha Kehrli, *Formalizing Grounded Arithmetic atop Isabelle/Pure*, BSc thesis, Distributed Computing Group, ETH Zürich, 2025.

2.1 Formulas and terms

Two Pure types carry the embedding. `o` is the type of QGA formulas and `num` is the type of QGA terms. A judgment declaration supplies the coercion `Trueprop :: o ⇒ prop` that lets a formula be asserted at the framework level, so that a QGA formula can appear as the conclusion of a Pure rule.

2.2 Structural rules

The rules in the rest of this section are written without a hypothesis context, because Pure maintains one. Making it visible for a moment, a QGA judgment is $\Gamma \vdash \varphi$ with Γ a finite set of formulas, and the framework supplies the operations that move judgments around.

$$\begin{array}{c} \frac{}{\Gamma, \varphi \vdash \varphi} \text{ (HYP)} \quad \frac{\Gamma \vdash \varphi}{\Gamma, \psi \vdash \varphi} \text{ (WEAK)} \quad \frac{\Gamma \vdash \psi \quad \Gamma, \psi \vdash \varphi}{\Gamma \vdash \varphi} \text{ (CUT)} \\[10pt] \frac{\Gamma \vdash \varphi}{[x \mapsto t]\Gamma \vdash [x \mapsto t]\varphi} \text{ (INST)} \end{array}$$

None of these is an axiom of `GD.thy`. `HYP` is Pure's `assume`, `WEAK` and `CUT` are introduction and elimination of $\Rightarrow$, and `INST` is instantiation of a schematic variable. A rule written $\llbracket P; Q \rrbracket \Rightarrow R$ therefore means the judgment $P, Q \vdash R$, and a theorem written that way holds for every instance of its free variables. Contraction and exchange do not appear at all, since Γ is a set.

One further operation has no sequent form. If $\Gamma \vdash \varphi$ is derived for a variable x that occurs nowhere in Γ , the derivation is uniform in x , and Pure records this as $\wedge x$. This is the eigenvariable condition, and it is what the premises of `IND` and `VI` below rely on.

2.3 Propositional rules

Disjunction and negation are primitive.

$$\begin{array}{c} \frac{P}{P \vee Q} \text{ (vI1)} \quad \frac{Q}{P \vee Q} \text{ (vI2)} \quad \frac{\neg P \quad \neg Q}{\neg(P \vee Q)} \text{ (vI3)} \\[10pt] \frac{\neg(P \vee Q)}{\neg P} \text{ (vE2)} \quad \frac{\neg(P \vee Q)}{\neg Q} \text{ (vE3)} \quad \frac{P \vee Q \quad \begin{array}{c} [P] \\ \vdots \\ R \end{array} \quad \begin{array}{c} [Q] \\ \vdots \\ R \end{array}}{R} \text{ (vE1)} \\[10pt] \frac{P}{\neg \neg P} \text{ (}\neg\neg\text{I)} \quad \frac{\neg \neg P}{P} \text{ (}\neg\neg\text{E)} \quad \frac{P \quad \neg P}{Q} \text{ (exF)} \end{array}$$

```
typedec1 o

judgment
  Trueprop :: <o ⇒ prop>  (<_> 5)

axiomatization
  disj :: <o ⇒ o ⇒ o>  (infixr <v> 30) and
  not  :: <o ⇒ o>  (<¬ _> [40] 40)where
  disjI1: <P ⇒ P v Q> and
  disjI2: <Q ⇒ P v Q> and
  disjI3: <[¬P; ¬Q] ⇒ ¬(P v Q)> and
  disjE1: <[P v Q; P ⇒ R; Q ⇒ R] ⇒ R> and
```

```

disjE2: <¬(P ∨ Q) ⇒ ¬P> and
disjE3: <¬(P ∨ Q) ⇒ ¬Q> and
dNegI: <P ⇒ (¬¬P)> and
dNegE: <(¬¬P) ⇒ P> and
exF: <[[P; ¬P]] ⇒ Q>

```

The introduction rules are the classical ones and the De Morgan directions $\vee E2$ and $\vee E3$ are available unconditionally. What is weakened is case analysis. $\vee E1$ needs an actual proof of $P \vee Q$, and $P \vee Q$ is not derivable for arbitrary P : the law of excluded middle is a judgment to be established rather than an axiom, and it is written $P \text{ B}$.

2.4 Equality and the habeas quid judgments

Equality is a binary predicate on terms, $\text{num} \Rightarrow \text{num} \Rightarrow o$, axiomatized by substitution and symmetry.

$$\frac{a = b \quad Qa}{Qb} \text{ (=E)} \qquad \frac{a = b}{b = a} \text{ (=S)}$$

```

eq :: <num ⇒ num ⇒ o> (infixl <=> 45)
where
  eqSubst: <[[a = b; Q a]] ⇒ Q b> and
  eqSym: <a = b ⇒ b = a> and
  eq_reflection: <x = y ⇒ x ≡ y>

```

Reflexivity is absent, and this is the pivot of the whole system. Transitivity follows from one substitution, eq_trans : $a = b \Rightarrow b = c \Rightarrow a = c$. The third axiom, eq_reflection , lifts a derivable equation to the framework equality $\equiv$, which is what lets Isabelle's simplifier operate on QGA goals; it is a statement about the framework and adds nothing to the object logic.

The two habeas quid judgments and the remaining connectives are definitions.

```

definition neq :: <num ⇒ num ⇒ o> (infixl <≠> 45)
  where <a ≠ b ≡ ¬ (a = b)>
definition bJudg :: <o ⇒ o> (<_ B> [21] 20)
  where <(p B) ≡ (p ∨ ¬p)>
definition isNat :: <num ⇒ o> (<_ N> [21] 20)
  where "x N ≡ x = x"
definition conj :: <o ⇒ o ⇒ o> (infixl <∧> 35)
  where <p ∧ q ≡ ¬(¬p ∨ ¬q)>
definition impl :: <o ⇒ o ⇒ o> (infixr <→> 25)
  where <p → q ≡ ¬p ∨ q>
definition iff :: <o ⇒ o ⇒ o> (infixl <↔> 25)
  where <p ↔ q ≡ (p → q) ∧ (q → p)>

```

Definition 2.1 (Habeas quid). $a \text{ N}$, defined as $a = a$, asserts that the term a denotes a natural number, equivalently that the computation it names terminates. $p \text{ B}$, defined as $p \vee \neg p$, asserts that the formula p is grounded.

Since $a \text{ N}$ unfolds to $a = a$, reflexivity of equality says exactly that every term terminates, which is why it cannot be an axiom of a logic with unrestricted recursion. Conjunction, implication and the biconditional are defined from $\vee$ and $\neg$ in the classical way, and their rules are then derived. Conjunction comes out unconditional, while implication and the biconditional acquire habeas quid premises, because their derivations pass through $\vee E1$.

$$\begin{array}{c}
\frac{p \quad q}{p \wedge q} (\wedge I) \qquad \frac{p \wedge q}{p} (\wedge E1) \qquad \frac{p \wedge q}{q} (\wedge E2) \\
\\
\frac{a \text{ B } \begin{array}{c} [a] \\ \vdots \\ b \end{array}}{a \longrightarrow b} (\rightarrow I) \qquad \frac{a \longrightarrow b \quad a}{b} (\rightarrow E) \qquad \frac{a \text{ B } \begin{array}{c} [a] \\ \vdots \\ b \end{array} \quad b \text{ B } \begin{array}{c} [b] \\ \vdots \\ a \end{array}}{a \longleftrightarrow b} (\leftrightarrow I)
\end{array}$$

Two further derived rules are used constantly. Case analysis on a grounded formula,

$$\text{cases_bool: } \llbracket q \text{ B}; q \Rightarrow p; \neg q \Rightarrow p \rrbracket \Rightarrow p,$$

and grounded proof by contradiction,

$$\text{grounded_contradiction: } \llbracket p \text{ B}; \neg p \Rightarrow q; \neg p \Rightarrow \neg q \rrbracket \Rightarrow p, \quad \text{contradiction: } \llbracket p \text{ B}; \neg p \Rightarrow \text{False} \rrbracket \Rightarrow p.$$

Both require the formula in question to be grounded. The final consistency argument of §4.5 is an application of contradiction, and the groundedness premise it needs is the reason the abstract development assumes that proof checking is boolean.

2.5 Natural numbers

$$\begin{array}{c}
\frac{}{0 \text{ N}} (0I) \qquad \frac{S(a) = S(b)}{a = b} (S\text{-}INJ) \qquad \frac{a = b}{S(a) = S(b)} (S\text{-}CONG) \\
\\
\frac{a = b}{P(a) = P(b)} (P\text{-}CONG) \qquad \frac{a \text{ N} \quad b \text{ N}}{(a = b) \text{ B}} (EQ\text{BOOL}) \qquad \frac{a \text{ N}}{S(a) \neq 0} (S \neq 0) \\
\\
\frac{a \text{ N}}{P(S(a)) = a} (PS) \qquad \frac{}{P(0) = 0} (P0) \qquad \frac{(a = b) \text{ B}}{a \text{ N} \wedge b \text{ N}} (EQE) \\
\\
\frac{P(a) \text{ N}}{a \text{ N}} (PREDTI) \qquad \frac{a \text{ N} \quad Q0 \quad \begin{array}{c} [x \text{ N}, Qx] \\ \vdots \\ QS(x) \end{array}}{Qa} (IND)
\end{array}$$

```

zero :: <num>                                and
suc  :: <num => num>      (<S(_)> [800]) and
pred :: <num => num>      (<P(_)> [800])
where
  nat0: <zero N> and
  sucInj: <S a = S b => a = b> and
  sucCong: <a = b => S a = S b> and
  predCong: <a = b => P a = P b> and
  eqBool: <[a N; b N] => (a = b) B> and
  (*eqBoolB: <[x B; y B] => (x = y) B> and*)
  sucNonZero: <a N => S a ≠ zero> and
  predSucInv: <a N => P(S(a)) = a> and
  pred0: <P(zero) = zero> and
  eqE: <((a = b) B) => ((a N) ∧ (b N))> and
  predTIE: <(P a N) => (a N)> and
  ind [case_names HQ Base Step]:
    "[a N; Q zero; ∧x. x N => Q x => Q S(x)] => Q a"

```

eqBool is the rule that converts termination into decidability: two terminating terms are either equal or not. eqE and predTIE run in the opposite direction, recovering termination from

groundedness of an equation and from termination of a predecessor. Induction carries a habeas quid premise on the subject of the induction, and its step case may assume both x N and the induction hypothesis.

Truth and falsity are definitions.

```
definition True
  where <True  $\equiv$  zero = zero>
definition False
  where <False  $\equiv$  S(zero) = zero>
```

Derived facts of constant use are `zeroRefl`: `zero = zero`, `natS`: `a N  $\implies$  S a N`, `natSI`: `S a N  $\implies$  a N`, `natP`: `a N  $\implies$  P a N`, and the pair `eq_impl_term`: `a = b  $\implies$  a N` and `eq_impl_term2`: `a = b  $\implies$  b N`, by which a derivable equation witnesses the termination of both of its sides.

2.6 Quantifiers

$$\begin{array}{c}
\begin{array}{c} [x\ N] \\ \vdots \\ Qx \\ \hline \forall x. Qx \end{array} \quad (\forall I) \qquad \begin{array}{c} \forall x. Qx \quad a\ N \\ \hline Qa \end{array} \quad (\forall E) \\
\\
\begin{array}{c} a\ N \quad Qa \\ \hline \exists x. Qx \end{array} \quad (\exists I) \qquad \begin{array}{c} [a\ N, Qa] \\ \vdots \\ \exists x. Qx \quad R \\ \hline R \end{array} \quad (\exists E)
\end{array}$$

```
forall :: "(num  $\Rightarrow$  o)  $\Rightarrow$  o" (binder "∀" [8] 9) and
exists :: "(num  $\Rightarrow$  o)  $\Rightarrow$  o" (binder "∃" [8] 9)
where
  forallI: "[[ $\wedge x. x\ N \implies Q\ x$ ]]  $\implies$   $\forall x. Q\ x$ " and
  forallE: "[[ $\forall c'. Q\ c'; a\ N$ ]]  $\implies$   $Q\ a$ " and
  existsI: "[[ $a\ N; Q\ a$ ]]  $\implies$   $\exists x. Q\ x$ " and
  existsE: "[[ $\exists i. Q\ i; \wedge a. a\ N \implies Q\ a \implies R$ ]]  $\implies$   $R$ "
```

Both quantifiers range over `num` and are relativized to terminating witnesses. $\forall I$ proves the body for an arbitrary terminating eigenvariable, $\forall E$ instantiates only at terminating terms, and $\exists I$ must exhibit a terminating witness.

Under these four rules a negated universal is inert. `forall` is a primitive constant, so no elimination rule matches $\neg(\forall x. Qx)$: the De Morgan rules $\vee E2$ and $\vee E3$ want a negated disjunction, $\neg\neg E$ wants a double negation, and `exF` would first need $\forall x. Qx$ itself, which $\forall I$ supplies only from a uniform derivation. Nor does any rule conclude $\neg(\forall x. Qx)$ from a counterexample. Two further axioms supply both directions.

$$\begin{array}{c}
\begin{array}{c} a\ N \quad \neg(Qa) \\ \hline \neg(\forall x. Qx) \end{array} \quad (\neg\forall I) \qquad \begin{array}{c} [a\ N, \neg(Qa)] \\ \vdots \\ \neg(\forall x. Qx) \quad R \\ \hline R \end{array} \quad (\neg\forall E)
\end{array}$$

```
axiomatization where
  notForallI: "[[ $a\ N; \neg(F\ a)$ ]]  $\implies$   $\neg(\forall x. F\ x)$ " and
  notForallE: "[[ $\neg(\forall x. F\ x); \wedge a. a\ N \implies \neg(F\ a) \implies R$ ]]  $\implies$   $R$ "
```


$\neg\forall I$ refutes a universal from one terminating counterexample and $\neg\forall E$ extracts one, so together they restore the duality between $\neg\forall$ and $\exists\neg$ as a pair of inference rules. Each carries a habeas quid premise on the witness, which keeps them in line with the rest of the system: the counterexample has to be an actual number rather than a description of one. §7.3 gives the semantic clause that justifies them. They correspond to the rules taken as primitive in reflective grounded arithmetic, which are primitive there for the same reason.

The dual pair for the existential needs no axiom. From $\neg(\exists x. Q x)$, together with groundness of Q at the point under consideration, $\exists I$ manufactures the positive partner and exF closes the case. This gives

$$\text{existsNeg: } \llbracket \neg(\exists x. Q x); \wedge x. x N \Rightarrow (Q x) B \rrbracket \Rightarrow \forall x. \neg(Q x)$$

as a derived rule. The asymmetry is the one just described. $\exists$ has an introduction rule that applies to a local assumption, and $\forall$ does not.

Where the new axioms are used is worth recording, since it is a single place. §4 needs to decide whether a list of hypotheses is satisfied, and the negative case of that decision is the refutation of a universal. Nothing else in the development appeals to either rule.

2.7 Conditionals

The if-zero-else construct is written `if c then a else b` and is polymorphic in the branches, so it serves both for term-valued and for formula-valued conditionals.

$$\begin{array}{c} \frac{c \quad a N}{(\text{if } c \text{ then } a \text{ else } b) = a} \text{ (COND I1)} \qquad \frac{\neg c \quad b N}{(\text{if } c \text{ then } a \text{ else } b) = b} \text{ (COND I2)} \\ \frac{c \quad (\text{if } c \text{ then } a \text{ else } b) N}{a N} \text{ (COND E1)} \qquad \frac{(\text{if } c \text{ then } a \text{ else } b) N}{c B} \text{ (COND E3)} \\ \frac{c \quad Q(\text{if } c \text{ then } a \text{ else } b)}{Q a} \text{ (COND-THEN-E)} \qquad \frac{c \quad Q a}{Q(\text{if } c \text{ then } a \text{ else } b)} \text{ (COND-THEN-I)} \end{array}$$

```
cond :: <o => 'a => 'a => 'a> (<if _ then _ else _> [25, 24, 24] 24)

axiomatization where
condI1: <llbracket c; a N &rrbracket => (if c then a else b) = a> and
condI2: <llbracket ¬c; b N &rrbracket => (if c then a else b) = b> and
(*condT: <llbracket c B; a N; b N &rrbracket => if c then a else b N> and*)
condI1B: <llbracket c; d B &rrbracket => (if c then d else e) ↔ d> and
condI2B: <llbracket ¬c; e B &rrbracket => (if c then d else e) ↔ e> and
(*condTB: <llbracket c B; d B; e B &rrbracket => if c then d else e B>*)
(*New additions*)
condE1: <llbracket c; (if c then a else b) N &rrbracket => (a N)> and
condE2: <llbracket ¬c; (if c then a else b) N &rrbracket => (b N)> and
condE3: <llbracket (if c then a else b) N &rrbracket => (c B)> and
condE1B: <llbracket c; (if c then d else e) B &rrbracket => (d B)> and
condE2B: <llbracket ¬c; (if c then d else e) B &rrbracket => (e B)> and
condE3B: <llbracket (if c then d else e) B &rrbracket => (c B)> and
(* lazy conditional rules *)
cond_thenQ_E: "c => Q (if c then a else b) => Q a" and
cond_thenQ_I: "c => Q a => Q (if c then a else b)" and
cond_elseQ_E: "¬ c => Q (if c then a else b) => Q b" and
cond_elseQ_I: "¬ c => Q b => Q (if c then a else b)"
```

Each rule has a formula-level counterpart in which $=$ is replaced by $\leftrightarrow$ and N by B , and each “then” rule has an “else” twin. The last pair is the lazy form: once the guard is decided, an

arbitrary context Q transports across the conditional. These are the rules that let a recursive definition be unfolded one branch at a time without evaluating the other branch. The two totality lemmas

$$\text{condT: } \llbracket c \ B; a \ N; b \ N \rrbracket \Rightarrow (\text{if } c \text{ then } a \text{ else } b) \ N, \quad \text{condTB: } \llbracket c \ B; a \ B; b \ B \rrbracket \Rightarrow (\text{if } c \text{ then } a \text{ else } b) \ B$$

are derived rather than assumed, and they carry most of the groundedness proofs about the encoded checkers of §3.

2.8 The definitional mechanism

A recursive definition is admitted with no termination side condition. The constant is introduced by axiomatization and its defining equation is an axiom of the form $f \ x := \text{body}$.

$$\frac{a := b \quad Qb}{Qa} \text{ (DEFE)} \quad \frac{a := b \quad Qa}{Qb} \text{ (DEFI)}$$

```
def :: <'a ⇒ 'a ⇒ o> (infix <:=> 10)
where
  defE: <llbracket a := b; Q b \rrbracket ⇒ Q a> and
  defI: <llbracket a := b; Q a \rrbracket ⇒ Q b>
```

$a := b$ does not assert $a = b$. It asserts that a and b are interchangeable in every context Q , whether or not either denotes a value. That is what makes $\omega := \omega$ harmless: ω is interchangeable with itself and $\omega \ N$ is underivable. The two rules are the unfolding and folding directions, and the `unfold_def` method of `unfold_def.ML` applies them.

2.9 Arithmetic

The standard functions are introduced by exactly this mechanism.

```
add    :: "num ⇒ num ⇒ num" (infixl "+" 60) and
sub    :: "num ⇒ num ⇒ num" (infixl "-" 60) and
mult   :: "num ⇒ num ⇒ num" (infixl "*" 70) and
div    :: "num ⇒ num ⇒ num"
less   :: "num ⇒ num ⇒ num" (infix "<" 50) and
leq    :: "num ⇒ num ⇒ num" (infix "≤" 50) and
omega  :: "'a"
where
  add_def:  "add x y := if y = 0 then x else S(add x (P y))" and
  sub_def:  "sub x y := if y = 0 then x else P(sub x (P y))" and
  mult_def: "mult x y := if y = 0 then 0 else (x + mult x (P y))" and
  leq_def:  "leq x y := if x = 0 then 1
              else if y = 0 then 0
              else (leq (P x) (P y))" and
  less_def: "less x y := if y = 0 then 0
              else if x = 0 then 1
              else (less (P x) (P y))" and
  div_def:  "div x y := if x < y = 1 then 0 else S(div (x - y) y)" and
  omega_def: "omega := omega"
```

Comparison returns a term rather than a formula, so $x \leq y$ evaluates to 1 or 0 and the associated formula is $x \leq y = 1$. The constant ω is included to exercise the claim that a divergent definition is admissible. Termination lemmas such as `add_terminates`, `leq_terminates` and `less_terminates` are proved by induction, as is termination of Ackermann's function, which is defined in the same file and is not primitive recursive.

2.10 Induction principles

Two derived induction principles carry most of the later work.

```
lemma strong_induction [consumes 1, case_names HQ Base Step]:
  shows "a N  $\implies$  Q 0  $\implies$  ( $\wedge x. x N \implies$  ( $\wedge y. y N \implies y \leq x = 1 \implies Q y$ )  $\implies$  (Q S(x)))  $\implies$  Q a"
```

```
lemma list_induct [case_names HQ Nil Cons]:
  "a N  $\implies$  Q Nil  $\implies$  ( $\wedge x xs. x N \implies xs N \implies Q xs \implies Q$  (Cons x xs))  $\implies$  Q a"
```

Strong induction is stated with the bounded hypothesis $y \leq x = 1$. It is what makes recursion over the structure of an encoded term legitimate: a component of a code is numerically smaller than the code, so structural recursion becomes bounded numerical recursion. `list_induct` is derived from it through the encoding of lists below.

2.11 Pairing

Pairing is the bridge from numbers to syntax. It is defined recursively rather than by a closed formula.

```
axiomatization cpair :: "num  $\Rightarrow$  num  $\Rightarrow$  num" where
  cpair_def: "cpair x y := if y = 0 then div (x * S(x)) 2
    else cpair x P(y) + x + y + 1"
```

The syntax $\langle x, y \rangle$ abbreviates `cpair x y` and nests to the right for tuples. The inverses are recursive as well.

```
cpx :: "num  $\Rightarrow$  num" and
cpy :: "num  $\Rightarrow$  num"
where
  cpx_def: "cpx x := if x = 0 then 0
    else if cpx (P x) = 0 then S(cpy P(x))
    else P(cpx (P x))" and
  cpy_def: "cpy x := if cpx (P x) = 0 then 0
    else S(cpy (P x))"
```

`cpi n x` projects the n th component of a right-nested tuple by iterating `cpy` and applying `cpx`.

```
cpi' :: "num  $\Rightarrow$  num  $\Rightarrow$  num"
where
  cpi'_def: "cpi' n x := if n = 0 then 0
    else if n = 1 then x
    else cpy (cpi' (n-1) x)"
```

The properties established for the pairing are exactly the ones the encoding of §5 needs.

<code>cpair_terminates</code>	$x N \implies y N \implies \langle x, y \rangle N$
<code>cpair_suc</code>	$x N \implies y N \implies \langle x, S(y) \rangle = \langle x, y \rangle + x + S(y) + 1$
<code>cpx_proj, cpy_proj</code>	the projection equations
<code>cpair_inj</code>	injectivity
<code>cpair_surjective</code>	every number is a pair
<code>cpair_reconstr</code>	$a N \implies \langle cpx a, cpy a \rangle = a$
<code>cpx_mono, cpy_mono</code>	each projection is bounded by its argument

```

lemma cpx_proj [simp]: "a N  $\implies$  b N  $\implies$  cpx (a, b) = a"
  apply (rule conjE1)
  apply (rule cpx_cpy_proj)
  apply (assumption)+
  done

(* separated lemma for cpy*)
lemma cpy_proj [simp]: "a N  $\implies$  b N  $\implies$  cpy (a, b) = b"

```

```

lemma cpair_inj:
  assumes eq: "(a, b) = (c, d)"
  shows "a N  $\implies$  b N  $\implies$  c N  $\implies$  d N  $\implies$  a = c  $\wedge$  b = d"

```

```

lemma cpair_surjective [auto]:
  assumes a_nat : "a N"
  shows "  $\exists$  b c. a = (b,c)"

```

```

lemma cpy_mono [simp]: "x N  $\implies$  cpy x  $\leq$  x = 1"
  apply (induct x)
  proof (simp add: cpy_suc)+
    case (Step xa)
    show "x N  $\implies$  xa N  $\implies$  cpy xa  $\leq$  xa = 1  $\implies$ 
      (if cpx xa = 0 then 0 else S(cpy xa))  $\leq$  (S xa) = 1"
      by (cases bool: "cpx xa = 0", simp+)
  qed

lemma cpx_mono [simp]: "x N  $\implies$  cpx x  $\leq$  x = 1"

```

The Cantor pairing is a bijection between $\mathbb{N}^2$ and $\mathbb{N}$, and both directions of that fact are used later: injectivity gives the decomposition properties of the encoding, and surjectivity gives `cpair_reconstr`, which says that a number can always be taken apart and put back together. Monotonicity feeds strong induction: since `cpx x  $\leq$  x` and `cpy x  $\leq$  x`, a recursion that descends into components terminates.

2.12 Lists

Hypothesis sets, assignments, definition lists and proofs are all lists, so lists are the last piece of QGA infrastructure. `Nil` is 0 and `Cons` is a shifted pair, which makes the list encoding a bijection with $\mathbb{N}$ and removes the need for any well-formedness predicate on list codes.

```

type_synonym List = num

definition Nil :: "List" where
  "Nil  $\equiv$  0"

definition Cons :: "num  $\Rightarrow$  List  $\Rightarrow$  List" where
  "Cons n xs  $\equiv$  (n, xs) + 1"

definition list_hd :: "List  $\Rightarrow$  num" where
  "list_hd x  $\equiv$  cpx (P x)"

definition list_tl :: "List  $\Rightarrow$  List" where
  "list_tl x  $\equiv$  cpy (P x)"

```

The derived facts are the expected ones: `nil_nat, cons_nat:  $\llbracket n \text{ N}; xs \text{ N} \rrbracket \implies \text{Cons } n \text{ xs N}$` ,

disjointness of the two constructors, the projection equations $\text{list_hd } (\text{Cons } n \text{ } xs) = n$ and $\text{list_tl } (\text{Cons } n \text{ } xs) = xs$, reconstruction $\text{cons_reconstr}: \llbracket x \text{ } N; \neg(x = \text{Nil}) \rrbracket \Rightarrow \text{Cons } (\text{list_hd } x) (\text{list_tl } x) = x$, injectivity cons_inj , exhaustiveness $\text{list_cases}: x \text{ } N \Rightarrow (x = \text{Nil}) \vee (\exists n \text{ } xs. x = \text{Cons } n \text{ } xs)$, and list_induct . Together these are the characteristic properties of an inductive datatype, obtained with no datatype package.

Four list functions are defined recursively and used throughout §3.

```
axiomatization mem :: "num  $\Rightarrow$  List  $\Rightarrow$  o" (infixr "∈" 75) where
  mem_def: "mem x G := if G = Nil then False
              else if list_hd G = x then True
              else mem x (list_tl G)"
```

```
axiomatization nth :: "num  $\Rightarrow$  List  $\Rightarrow$  num" where
  nth_def: "nth i xs := if xs = Nil then 0
                      else if i = 0 then list_hd xs
                      else nth (i - 1) (list_tl xs)"
```

```
axiomatization len :: "List  $\Rightarrow$  num" where
  len_def: "len xs := if xs = Nil then 0 else S (len (list_tl xs))"
```

```
axiomatization subset :: "List  $\Rightarrow$  List  $\Rightarrow$  o" where
  subset_def: "subset G' G :=
              if G' = Nil then True
              else mem (list_hd G') G  $\wedge$  subset (list_tl G') G"
```

The two habeas quid results about them that the proof checker relies on are $\text{mem_bool}: \llbracket x \text{ } N; G \text{ } N \rrbracket \Rightarrow \text{mem } x \text{ } G \text{ } B$ and $\text{nth_in_range_N}: \llbracket xs \text{ } N; i < \text{len } xs = 1 \rrbracket \Rightarrow \text{nth } i \text{ } xs \text{ } N$.

2.13 Summary

Table 1 collects the definitions of `GD.thy` and Table 2 its inference rules, both in the notation of QGA rather than in Isabelle syntax. A definition written with $\triangleq$ is an abbreviation, unfolded by the framework. A definition written with $:=$ is a QGA recursive definition in the sense of §2.8, admitted with no termination obligation and used only through `DEFE` and `DEFL`.

3 BGA arithmetized in QGA

`BGA_on_GA.thy` encodes basic grounded arithmetic as QGA natural numbers and proves its proof checker sound with respect to the encoded semantics. The file is a tower of locales, so the whole argument is carried out under a fixed, small set of assumptions about the encoding and no concrete Gödel numbering appears in it.

This section walks through the locales in order and explains every constant they fix. Throughout, $\Gamma \vdash \varphi$ is a BGA judgment, π and ρ range over encoded proofs, A over assignments, and D over the background definition list. Notation like $q[v_i \mapsto a]$ abbreviates $\text{subst_F } q \text{ } i \text{ } a$.

3.1 The system being encoded

BGA has terms and formulas, and no logical connectives at all.

$$\begin{aligned} t &::= v_j \mid 0 \mid S(t) \mid P(t) \mid t \text{ } 0? t : t \mid d_i(t, t) \\ \varphi &::= t = t \mid t \neq t \end{aligned}$$

Abbreviations

$a \neq b$	$\triangleq \neg(a = b)$
$p \text{ B}$	$\triangleq p \vee \neg p$
$a \text{ N}$	$\triangleq a = a$
$p \wedge q$	$\triangleq \neg(\neg p \vee \neg q)$
$p \longrightarrow q$	$\triangleq \neg p \vee q$
$p \longleftrightarrow q$	$\triangleq (p \longrightarrow q) \wedge (q \longrightarrow p)$
True	$\triangleq 0 = 0$
False	$\triangleq S(0) = 0$
$x > y$	$\triangleq 1 - (x \leq y)$
$x \geq y$	$\triangleq 1 - (x < y)$
Nil	$\triangleq 0$
Cons $n \text{ xs}$	$\triangleq \langle n, \text{xs} \rangle + 1$
hd x	$\triangleq \text{cpx } P(x)$
tl x	$\triangleq \text{cpy } P(x)$

Recursive definitions

$x + y$	$:= \text{if } y = 0 \text{ then } x \text{ else } S(x + P(y))$
$x - y$	$:= \text{if } y = 0 \text{ then } x \text{ else } P(x - P(y))$
$x * y$	$:= \text{if } y = 0 \text{ then } 0 \text{ else } x + (x * P(y))$
$x \leq y$	$:= \text{if } x = 0 \text{ then } 1 \text{ else if } y = 0 \text{ then } 0 \text{ else } P(x) \leq P(y)$
$x < y$	$:= \text{if } y = 0 \text{ then } 0 \text{ else if } x = 0 \text{ then } 1 \text{ else } P(x) < P(y)$
div $x \ y$	$:= \text{if } (x < y) = 1 \text{ then } 0 \text{ else } S(\text{div } (x - y) \ y)$
ω	$:= \omega$
$\langle x, y \rangle$	$:= \text{if } y = 0 \text{ then } \text{div } (x * S(x)) \ 2 \text{ else } \langle x, P(y) \rangle + x + y + 1$
cpx x	$:= \text{if } x = 0 \text{ then } 0 \text{ else if cpx } P(x) = 0 \text{ then } S(\text{cpy } P(x)) \text{ else } P(\text{cpx } P(x))$
cpy x	$:= \text{if cpx } P(x) = 0 \text{ then } 0 \text{ else } S(\text{cpy } P(x))$
cpi' $n \ x$	$:= \text{if } n = 0 \text{ then } 0 \text{ else if } n = 1 \text{ then } x \text{ else cpy } (\text{cpi}' (n - 1) \ x)$
$x \in G$	$:= \text{if } G = \text{Nil} \text{ then } \text{False} \text{ else if hd } G = x \text{ then } \text{True} \text{ else } x \in \text{tl } G$
nth $i \ \text{xs}$	$:= \text{if } \text{xs} = \text{Nil} \text{ then } 0 \text{ else if } i = 0 \text{ then } \text{hd } \text{xs} \text{ else } \text{nth } (i - 1) \ (\text{tl } \text{xs})$
len xs	$:= \text{if } \text{xs} = \text{Nil} \text{ then } 0 \text{ else } S(\text{len } (\text{tl } \text{xs}))$
subset $G' \ G$	$:= \text{if } G' = \text{Nil} \text{ then } \text{True} \text{ else hd } G' \in G \wedge \text{subset } (\text{tl } G') \ G$
ack $x \ y$	$:= \text{if } x = 0 \text{ then } y + 1 \text{ else if } y = 0 \text{ then } \text{ack } P(x) \ 1 \text{ else } \text{ack } P(x) \ (\text{ack } x \ P(y))$

Table 1: Definitions of QGA. $\langle x, y \rangle$ is the Cantor pairing and cpx, cpy its projections.

Structural (supplied by the framework)

$$\frac{}{\Gamma, \varphi \vdash \varphi} \text{ (HYP)} \quad \frac{\Gamma \vdash \varphi}{\Gamma, \psi \vdash \varphi} \text{ (WEAK)} \quad \frac{\Gamma \vdash \psi \quad \Gamma, \psi \vdash \varphi}{\Gamma \vdash \varphi} \text{ (CUT)}$$

$$\frac{\Gamma \vdash \varphi}{[x \mapsto t]\Gamma \vdash [x \mapsto t]\varphi} \text{ (INST)}$$

Propositional

$$\frac{P}{P \vee Q} \text{ (}\vee\text{I1)} \quad \frac{Q}{P \vee Q} \text{ (}\vee\text{I2)} \quad \frac{\neg P \quad \neg Q}{\neg(P \vee Q)} \text{ (}\vee\text{I3)}$$

$$\frac{\neg(P \vee Q)}{\neg P} \text{ (}\vee\text{E2)} \quad \frac{\neg(P \vee Q)}{\neg Q} \text{ (}\vee\text{E3)} \quad \frac{P \vee Q \quad \begin{array}{c} [P] \\ \vdots \\ R \end{array} \quad \begin{array}{c} [Q] \\ \vdots \\ R \end{array}}{R} \text{ (}\vee\text{E1)}$$

$$\frac{P}{\neg\neg P} \text{ (}\neg\neg\text{I)} \quad \frac{\neg\neg P}{P} \text{ (}\neg\neg\text{E)} \quad \frac{P \quad \neg P}{Q} \text{ (EXF)}$$

Equality

$$\frac{a = b \quad Qa}{Qb} \text{ (=E)} \quad \frac{a = b}{b = a} \text{ (=S)}$$

Natural numbers

$$\frac{}{0 \text{ N}} \text{ (0I)} \quad \frac{a = b}{P(a) = P(b)} \text{ (P-CONG)} \quad \frac{a \text{ N}}{P(S(a)) = a} \text{ (PS)}$$

$$\frac{S(a) = S(b)}{a = b} \text{ (S-INJ)} \quad \frac{a \text{ N} \quad b \text{ N}}{(a = b) \text{ B}} \text{ (EQBOOL)} \quad \frac{a = b}{S(a) = S(b)} \text{ (S-CONG)}$$

$$\frac{a \text{ N}}{P(0) = 0} \text{ (P0)} \quad \frac{a \text{ N} \quad b \text{ N}}{a \text{ N} \wedge b \text{ N}} \text{ (EQE)} \quad \frac{a \text{ N}}{S(a) \neq 0} \text{ (S}\neq\text{0)}$$

$$\frac{P(a) \text{ N}}{a \text{ N}} \text{ (PREDTI)} \quad \frac{a \text{ N} \quad Q0 \quad \begin{array}{c} [x \text{ N}, Qx] \\ \vdots \\ Q S(x) \end{array}}{Qa} \text{ (IND)}$$

Quantifiers

$$\frac{\begin{array}{c} [x \text{ N}] \\ \vdots \\ Qx \end{array}}{\forall x. Qx} \text{ (}\forall\text{I)} \quad \frac{\forall x. Qx \quad a \text{ N}}{Qa} \text{ (}\forall\text{E)} \quad \frac{a \text{ N} \quad Qa}{\exists x. Qx} \text{ (}\exists\text{I)}$$

$$\frac{\exists x. Qx \quad \begin{array}{c} [a \text{ N}, Qa] \\ \vdots \\ R \end{array}}{R} \text{ (}\exists\text{E)} \quad \frac{a \text{ N} \quad \neg(Qa)}{\neg(\forall x. Qx)} \text{ (}\neg\forall\text{I)} \quad \frac{\neg(\forall x. Qx) \quad \begin{array}{c} [a \text{ N}, \neg(Qa)] \\ \vdots \\ R \end{array}}{R} \text{ (}\neg\forall\text{E)}$$

Conditionals

$$\frac{c \quad a \text{ N}}{(\text{if } c \text{ then } a \text{ else } b) = a} \text{ (COND I1)} \quad \frac{\neg c \quad b \text{ N}}{(\text{if } c \text{ then } a \text{ else } b) = b} \text{ (COND I2)}$$

$$\frac{c \quad (\text{if } c \text{ then } a \text{ else } b) \text{ N}}{a \text{ N}} \text{ (CONDE1)} \quad \frac{\neg c \quad (\text{if } c \text{ then } a \text{ else } b) \text{ N}}{b \text{ N}} \text{ (CONDE2)}$$

$$\frac{(\text{if } c \text{ then } a \text{ else } b) \text{ N}}{c \text{ B}} \text{ (CONDE3)} \quad \frac{c \quad Q \text{ (if } c \text{ then } a \text{ else } b)}{Qa} \text{ (COND-THEN-E)}$$

$$\frac{c \quad Qa}{Q \text{ (if } c \text{ then } a \text{ else } b)} \text{ (COND-THEN-I)} \quad \frac{\neg c \quad Qb}{Q \text{ (if } c \text{ then } a \text{ else } b)} \text{ (COND-ELSE-I)}$$

Definitions

$$\frac{a := b \quad Qb}{Qa} \text{ (DEF E)} \quad \frac{a := b \quad Qa}{Qb} \text{ (DEF I)}$$

A variable v_j is a natural number j , and its value under an assignment is the j th entry of the assignment list. BGA has no binders, so nothing scopes over a variable and the index is simply a position. Conditional evaluation is if-zero-else, which keeps terms and formulas from being mutually recursive. Function application is restricted to two arguments, and the functions come from a fixed background list D , where position i holds the body of $d_i(v_0, v_1)$. Disequality is a primitive, so $a \neq b$ is not the negation of $a = b$; BGA has no negation to form. A judgment is a sequent $\Gamma \vdash \varphi$ with Γ a finite list of formulas.

3.2 Codes

There is one QGA type, `num`. The type synonyms below are documentation.

```
type_synonym tm = num (* BGA Term *)
type_synonym fm = num (* BGA Formula *)
type_synonym asn = num (* Assignment *)
type_synonym dfn = num (* Definition *)
type_synonym pf = num (* Proof: List of judgements *)
type_synonym val = num (* Evaluated Value *)

type_synonym hyp = num (* Hypothesis: list/set of formulas *)
type_synonym jdg = num (* Judgment: (hyp, fm) *)

abbreviation hyp_of :: "jdg  $\Rightarrow$  hyp" where "hyp_of J  $\equiv$  cpx J"
abbreviation conc_of :: "jdg  $\Rightarrow$  fm" where "conc_of J  $\equiv$  cpy J"

abbreviation mk_jdg :: "hyp  $\Rightarrow$  fm  $\Rightarrow$  jdg" (infix " $\Vdash$ " 50)
  where "G  $\Vdash$  c  $\equiv$  (G, c)"
abbreviation emptyH :: "hyp" ("{}")
  where "{}  $\equiv$  Nil"
abbreviation cons :: "fm  $\Rightarrow$  List  $\Rightarrow$  List" (infixr " $\triangleright$ " 65)
  where "f  $\triangleright$  G  $\equiv$  Cons f G"
```

A judgment is the pair $\langle G, c \rangle$, written $G \Vdash c$, with `hyp_of` and `conc_of` as projections. A hypothesis set is a list of formulas and a proof is a list of judgments, read head first, each justified by the judgments after it.

3.3 The locale tower

Locale	Extends	Fixes
suff_syntax	—	mk_eq, mk_neq, dfns, is_valid_proof
suff_semantics	suff_syntax	evals, sat_fm, sat_hyp
consistent	suff_semantics	—
bga_encoding	—	the six encoder operations, the three freshness predicates
bga_dfns	bga_encoding	dfns, dfn_is
bga_fuel_semantics	bga_dfns	eval_fuel
bga_subst	bga_encoding	subst_T, subst_F, subst_body
bga_fuel_subst_semantics	bga_fuel_semantics, bga_subst	asn_put
bga_subst_rule	bga_subst	check_template, rep_vars_T, rep_vars_F, find_phi, find_eq, check_subst
bga_eq_rule	bga_encoding	check_eq_rules, check_neq_rules
bga_ind_rule	bga_subst_rule	check_ind_template, find_ind_base, check_ind
bga_struct_rule	bga_encoding	find_cut, check_cut, find_struct, check_struct
bga_app_rule	bga_subst_rule, bga_dfns	app_try, app_y, app_x, app_d, check_app
bga_proof_check	the five rule locales	valid_step, check_list, is_valid_proof
bga_fuller	bga_fuel_subst_semantics, bga_proof_check	—

Two locales are defined in the file and not used by `bga_fuller`: `bga_semantics`, which fixes the exact evaluator `eval`, and `bga_subst_semantics`, which combines it with substitution. They are described in §3.8 because the exact evaluator is the clearest statement of what the encoded semantics means.

Almost every constant is fixed with an assumption of the form $f\ x := \text{body}$ rather than being defined. A locale cannot make a recursive definition, and in QGA it does not need to: fixing a constant together with its $:=$ equation is exactly what §2.8 admits, and the equation is all the proofs ever use.

3.4 The consistency skeleton

Before any encoding is fixed, the shape of the final argument is stated.

```

locale suff_syntax =
  (*Encoding *)
  fixes mk_eq :: "tm  $\Rightarrow$  tm  $\Rightarrow$  fm"
  fixes mk_neq :: "tm  $\Rightarrow$  tm  $\Rightarrow$  fm"
  (* Fixed Definition List*)
  fixes dfns :: "dfn"
  (*Provability. is_valid_proof { p } { f }  $\equiv$  p { P } [  $\Gamma \vdash$ 
     $\hookrightarrow$  f ]*)
  fixes is_valid_proof :: "pf  $\Rightarrow$  jdg  $\Rightarrow$  o"

  (*Habeas Quid for Syntax Constructors *)
  assumes mk_eq_N: "[a N; b N]  $\Rightarrow$  mk_eq a b N"

```

```

assumes mk_neq_N: "[[a N; b N]]  $\Rightarrow$  mk_neq a b N"

(* Habeas Quid for the Proof Checker *)
assumes proof_bool: "[[p N; J N]]  $\Rightarrow$  ((is_valid_proof p J) B)"

```

mk_eq :: tm $\Rightarrow$ tm $\Rightarrow$ fm	builds the code of $a = b$ from the codes of a and b
mk_neq :: tm $\Rightarrow$ tm $\Rightarrow$ fm	the same for $a \neq b$
dfns :: dfn	the code of the background definition list D
is_valid_proof :: pf $\Rightarrow$ jdgc $\Rightarrow$ o	" π is a proof of J "

The assumptions are only habeas quid: encoded formulas are numbers, and `is_valid_proof p J` is a grounded formula whenever `p` and `J` are numbers. The second is the grounded-logic form of decidability of proof checking, and it is what makes the concluding proof by contradiction available.

```

locale suff_semantics = suff_syntax +
  (*Semantics. sat checks whether the encoding of a formula is satisfied by an assignment.
    $\hookrightarrow$  eval reduces a term under an assignment*)

  fixes evals :: "tm  $\Rightarrow$  asn  $\Rightarrow$  val  $\Rightarrow$  o"
  fixes sat_fm :: "fm  $\Rightarrow$  asn  $\Rightarrow$  o"
  fixes sat_hyp :: "hyp  $\Rightarrow$  asn  $\Rightarrow$  o"

  assumes sat_hyp_nil: "sat_hyp Nil A"

  (* Evaluation is deterministic *)
  assumes evals_det: "[[t N; A N; evals t A r; evals t A q]]  $\Rightarrow$  r = q"

  (* What equations mean in the model: the two sides evaluate to a common value
   (resp. to distinct values). *)
  assumes sat_eqE:
    "[[a N; b N; sat_fm (mk_eq a b) A;
      $\wedge$ q. [[q N; evals a A q; evals b A q]]  $\Rightarrow$  R]]  $\Rightarrow$  R"
  assumes sat_neqE:
    "[[a N; b N; sat_fm (mk_neq a b) A;
      $\wedge$ x y. [[x N; y N; evals a A x; evals b A y; x  $\neq$  y]]  $\Rightarrow$  R]]  $\Rightarrow$  R"

```

evals :: tm $\Rightarrow$ asn $\Rightarrow$ val $\Rightarrow$ o	" t evaluates to r under A "
sat_fm :: fm $\Rightarrow$ asn $\Rightarrow$ o	"the formula coded by f holds under A "
sat_hyp :: hyp $\Rightarrow$ asn $\Rightarrow$ o	"every formula in Γ holds under A "

Three assumptions: the empty hypothesis list is satisfied by every assignment, evaluation is deterministic, and a satisfied encoded equation yields a common value for its two sides while a satisfied encoded disequality yields two distinct values.

```

locale consistent = suff_semantics +
  (* Valid proofs yield satisfied formulas *)
  assumes soundness: "[[is_valid_proof p J; sat_hyp (hyp_of J) A; A N]]  $\Rightarrow$  sat_fm (conc_of J) A"
begin

lemma syntactically_consistent:
  assumes a_nat: "a N"
  assumes b_nat: "b N"
  assumes p1_nat: "p1 N"
  assumes p2_nat: "p2 N"
  shows "  $\neg$  (is_valid_proof p1 (Nil, (mk_eq a b))  $\wedge$  is_valid_proof p2 (Nil, (mk_neq a b)))"

```

consistent adds soundness of the checker and derives syntactic consistency. The derivation is short. Assume both proofs exist. The statement is grounded by `proof_bool`, so contradiction applies. Soundness under the empty hypothesis list and the assignment $\emptyset$ gives `sat_fm (mk_eq a b)  $\emptyset$` and `sat_fm (mk_neq a b)  $\emptyset$` . The first produces a value q reached by both a and b , the second produces x and y with $x \neq y$, determinism forces $x = q = y$, and `exF` closes the goal. The `habeas quid` premises $a \ N, b \ N, p1 \ N$ and $p2 \ N$ are needed before the formula being contradicted can be shown grounded.

3.5 Encoded syntax

Terms and formulas are tagged pairs, with six term tags and two formula tags.

```
abbreviation T_VAR  :: num where "T_VAR  $\equiv$  0"
abbreviation T_ZERO :: num where "T_ZERO  $\equiv$  1"
abbreviation T_SUC  :: num where "T_SUC  $\equiv$  2"
abbreviation T_PRED :: num where "T_PRED  $\equiv$  3"
abbreviation T_IFZ  :: num where "T_IFZ  $\equiv$  4"
abbreviation T_APP  :: num where "T_APP  $\equiv$  5"

(* Formula Constructor Tags *)
abbreviation F_EQ   :: num where "F_EQ  $\equiv$  0"
abbreviation F_NEQ  :: num where "F_NEQ  $\equiv$  1"
```

Tag	Constructor	Notation	Payload
T_VAR	variable	v_j	j
T_ZERO	zero	0	ignored
T_SUC	successor	$S(a)$	a
T_PRED	predecessor	$P(a)$	a
T_IFZ	if-zero-else	$c \ 0? a : b$	$\langle c, \langle a, b \rangle \rangle$
T_APP	application	$d_i(a, b)$	$\langle i, \langle a, b \rangle \rangle$
F_EQ	equality	$a = b$	$\langle a, b \rangle$
F_NEQ	disequality	$a \neq b$	$\langle a, b \rangle$

3.6 bga_encoding

```
locale bga_encoding =
  (*Term Encoding *)
  (* Tag of the current Constructor *)
  fixes tag_T :: "tm  $\Rightarrow$  tmtag"
  (* Encoding of the argument *)
  fixes load_T :: "tm  $\Rightarrow$  tm"
  (* Encodes the term *)
  fixes pack_T :: "tmtag  $\Rightarrow$  tm  $\Rightarrow$  tm"

  (* Formula Encoding *)
  fixes tag_F :: "fm  $\Rightarrow$  fmtag"
  fixes load_F :: "fm  $\Rightarrow$  tm"
  fixes pack_F :: "fmtag  $\Rightarrow$  fm  $\Rightarrow$  fm"

  (* Fixing Definition*)
  fixes fresh_T :: "num  $\Rightarrow$  tm  $\Rightarrow$  o"
  fixes fresh_F :: "num  $\Rightarrow$  fm  $\Rightarrow$  o"
  fixes fresh_H :: "num  $\Rightarrow$  hyp  $\Rightarrow$  o"

  assumes fresh_T_def:
```

```

"fresh_T k t :=
  if tag_T t = T_VAR then load_T t < k = 1
  else if tag_T t = T_ZERO then True
  else if tag_T t = T_SUC then fresh_T k (load_T t)
  else if tag_T t = T_PRED then fresh_T k (load_T t)
  else if tag_T t = T_IFZ then
    fresh_T k (cpx (load_T t)) ∧
    fresh_T k (cpx (cpy (load_T t))) ∧
    fresh_T k (cpy (cpy (load_T t)))
  else if tag_T t = T_APP then
    fresh_T k (cpx (cpy (load_T t))) ∧
    fresh_T k (cpy (cpy (load_T t)))
  else False"
assumes fresh_F_def: "fresh_F k f := fresh_T k (cpx (load_F f)) ∧ fresh_T k (cpy (load_F f))"
assumes fresh_H_def: "fresh_H k G := if G = Nil then True else fresh_F k (list_hd G) ∧
  ↪ fresh_H k (list_tl G)"

```

tag_T :: tm ⇒ tmtag	the constructor tag of a term code
load_T :: tm ⇒ tm	the payload of a term code
pack_T :: tmtag ⇒ tm ⇒ tm	builds a term code from tag and payload
tag_F, load_F, pack_F	the same three for formula codes
fresh_T :: num ⇒ tm ⇒ o	every variable in the term has index below k
fresh_F :: num ⇒ fm ⇒ o	the same for a formula
fresh_H :: num ⇒ hyp ⇒ o	the same for every formula in a list

Freshness is defined by recursion over the tag, and in mathematical notation it reads

$$\text{fresh}_T(k, t) = \begin{cases} j < k & t = v_j \\ T & t = 0 \\ \text{fresh}_T(k, a) & t = S(a) \text{ or } t = P(a) \\ \text{fresh}_T(k, c) \wedge \text{fresh}_T(k, a) \wedge \text{fresh}_T(k, b) & t = (c \ 0? a : b) \\ \text{fresh}_T(k, a) \wedge \text{fresh}_T(k, b) & t = d_i(a, b) \\ \perp & \text{tag out of range} \end{cases}$$

with $\text{fresh}_F(k, \varphi)$ the conjunction over the two sides of φ and $\text{fresh}_H(k, \Gamma)$ the conjunction over the members of Γ . The last line is the treatment of a number whose tag is not one of the six: it is fresh for no k . Freshness is used once, in the induction rule of §3.14, to guarantee that the induction variable does not occur in the hypotheses.

The assumptions on the interface are the following, and they are the only properties of the encoding used anywhere in the development.

```

assumes pack_F_N: "[[t N; L N]] ⇒ pack_F t L N"
assumes tag_F_N: "f N ⇒ tag_F f N"
assumes load_F_N: "f N ⇒ load_F f N"

assumes pack_T_N: "[[t N; L N]] ⇒ (pack_T t L) N"
assumes tag_T_N: "t N ⇒ tag_T t N"
assumes load_T_N: "t N ⇒ load_T t N"

(*Injective *)
assumes tag_pack_F: "[[t N; L N]] ⇒ tag_F (pack_F t L) = t"
assumes load_pack_F: "[[t N; L N]] ⇒ load_F (pack_F t L) = L"

assumes tag_pack_T: "[[t N; L N]] ⇒ tag_T (pack_T t L) = t"
assumes load_pack_T: "[[t N; L N]] ⇒ load_T (pack_T t L) = L"

```

```

(*Surjective *)
assumes pack_tag_F: "f N  $\implies$  pack_F (tag_F f) (load_F f) = f"
assumes pack_tag_T: "t N  $\implies$  pack_T (tag_T t) (load_T t) = t"

(*Structural Decrease (For Induction) *)
(* This will require load of atomic terms(without arguments) to return 0 *)
assumes decrease_F: "[[f N; f  $\neq$  0]]  $\implies$  load_F f < f = 1"
assumes decrease_T: "[[t N; t  $\neq$  0]]  $\implies$  load_T t < t = 1"

assumes tag_T_zero: "tag_T 0 = T_ZERO"

(*Monotone assumptions for Bounding *)
assumes mono_pack_T: "[[tg N; x N; y N; x  $\leq$  y = 1]]  $\implies$  (pack_T tg x  $\leq$  pack_T tg y = 1)"
assumes mono_pack_F: "[[tg N; x N; y N; x  $\leq$  y = 1]]  $\implies$  (pack_F tg x  $\leq$  pack_F tg y = 1)"

```

habeas quid	pack, tag and load preserve N
injectivity	tag (pack t L) = t and load (pack t L) = L
retraction	pack (tag f) (load f) = f
decrease	$f \neq 0 \implies \text{load } f < f = 1$, payloads are strictly smaller
base case	tag_T 0 = T_ZERO, the code 0 is the term 0
monotonicity	pack is monotone in its payload

Remark 3.1 (Injective, not surjective). What the concrete encoding of §5 provides, and all that is used, is an *injective* map from BGA syntax into $\mathbb{N}$. It is not surjective: a number whose tag lies outside the constructor range encodes no BGA term, and no attempt is made to exclude such numbers or to recognize the well-formed ones. The assumption that does hold for every number is the retraction property $\text{pack } (\text{tag } f) (\text{load } f) = f$, which follows from the Cantor pairing being a bijection on $\mathbb{N}$ and says only that a code can be taken apart and put back together. Ill-formed codes are harmless: the evaluator gives them the application semantics, freshness rejects them, and soundness never needs them to denote anything.

The decrease and monotonicity assumptions are what allow `strong_induction` to serve as structural induction over encoded terms. Decrease gives the descent: the payload of a nonzero code is a strictly smaller number, so recursion into subterms is bounded numerical recursion. Monotonicity is used differently, to justify the bounds of the searches in §3.12. The first lemma proved from them is `fresh_T_bool`, that freshness is a grounded formula.

3.7 bga_dfns

```

locale bga_dfns = bga_encoding +
  fixes dfns :: "dfn"
  fixes dfn_is :: "dfn  $\Rightarrow$  num  $\Rightarrow$  tm  $\Rightarrow$  o"
  assumes dfns_N: "dfns N"
  (* lazy range check instea of direct conjunction which makes it kind of awkward to work with
     *)
  assumes dfn_is_def: "dfn_is d k b := if d < len dfns = 1 then nth d dfns = b  $\wedge$  fresh_T k b
     else False"

```

dfns :: dfn	the code of the definition list D , a list of term codes
dfn_is :: dfn $\Rightarrow$ num $\Rightarrow$ tm $\Rightarrow$ o	" d is in range, D_d is b , and b has arity $\leq k$ "

Written out, $\text{dfn_is}(d, k, b)$ holds when $d < |D|$, $D_d = b$ and $\text{fresh}_T(k, b)$. The range test is written as a nested conditional rather than a conjunction so that $\text{nth } d D$ is never mentioned when d is out of range. The application rule instantiates it at $k = 2$, which is the sense in which BGA definitions are binary: the body may mention v_0 and v_1 and nothing else.

3.8 Semantics

Two evaluators appear in the file. The direct one, fixed by `bga_semantics`, is the natural reading of the BGA big-step semantics, and writing it down at all depends on QGA tolerating divergent definitions.

```

assumes eval_def: "eval t A :=
  if tag_T t = T_VAR then nth (load_T t) A
  else if tag_T t = T_ZERO then 0
  else if tag_T t = T_SUC then S(eval (load_T t) A)
  else if tag_T t = T_PRED then P(eval (load_T t) A)
  else if tag_T t = T_IFZ then
    (if eval (cpx (load_T t)) A = 0
     then eval (cpx (cpy (load_T t))) A
     else eval (cpy (cpy (load_T t))) A)
  else
    (if eval (cpx (cpy (load_T t))) A = eval (cpx (cpy (load_T t))) A
     then (if eval (cpy (cpy (load_T t))) A = eval (cpy (cpy (load_T t))) A
            then eval (nth (cpx (load_T t)) dfns)
                  ((eval (cpx (cpy (load_T t))) A) > ((eval (cpy (cpy (load_T t))) A) > Nil))
            else 0)
     else 0)"

```

In mathematical notation, with A an assignment,

$$\text{eval}(t, A) = \begin{cases} \text{nth } j \ A & t = v_j \\ 0 & t = 0 \\ S(\text{eval}(a, A)) & t = S(a) \\ P(\text{eval}(a, A)) & t = P(a) \\ \text{eval}(a, A) \text{ or } \text{eval}(b, A) & t = (c \ 0? \ a : \ b), \text{ on } \text{eval}(c, A) = 0 \\ \text{eval}(D_i, [\text{eval}(a, A), \text{eval}(b, A)]) & t = d_i(a, b) \end{cases}$$

The application case in the sources is wrapped in two guards of the form `eval a A = eval a A`. Those are habeas quid tests: the arguments are evaluated first, and the body is entered only if both terminate, which makes the encoded semantics call by value.

The consistency argument runs through the step-indexed variant instead, fixed by `bga_fuel_semantics`. Its result is offset by one, so that `eval_fuel(k, t, A) = 0` means “no result within budget k ” and `eval_fuel(k, t, A) = 1 + v` means “the value is v ”. The offset is what lets a single num carry both outcomes, at the cost of an explicit `P(·)` on every recursive result.

```

locale bga_fuel_semantics = bga_dfns +
  fixes eval_fuel :: "num => tm => asn => val"
  assumes eval_fuel_def: "eval_fuel k t A :=
    if k = 0 then 0
    else if tag_T t = T_VAR then S (nth (load_T t) A)
    else if tag_T t = T_ZERO then S 0
    else if tag_T t = T_SUC then
      if eval_fuel (P k) (load_T t) A = 0 then 0
      else S (S (P (eval_fuel (P k) (load_T t) A)))
    else if tag_T t = T_PRED then
      if eval_fuel (P k) (load_T t) A = 0 then 0
      else S (P (P (eval_fuel (P k) (load_T t) A)))
    else if tag_T t = T_IFZ then
      if eval_fuel (P k) (hyp_of (load_T t)) A = 0 then 0
      else if P (eval_fuel (P k) (hyp_of (load_T t)) A) = 0
         then eval_fuel (P k) (hyp_of (conc_of (load_T t))) A
      else eval_fuel (P k) (conc_of (conc_of (load_T t))) A
    else

```

```

if eval_fuel (P k) (hyp_of (conc_of (load_T t))) A = 0 then 0
else if eval_fuel (P k) (conc_of (conc_of (load_T t))) A = 0 then 0
else eval_fuel (P k) (nth (hyp_of (load_T t)) dfns)
(P (eval_fuel (P k) (hyp_of (conc_of (load_T t))) A) >
 P (eval_fuel (P k) (conc_of (conc_of (load_T t))) A) > Nil)"

```

```

definition evals :: "tm ⇒ asn ⇒ val ⇒ o" where
  "evals t A r ≡ ∃k. eval_fuel k t A = S r"

```

`evals t A r` says that t evaluates to r under A for some budget:

$$\text{evals}(t, A, r) \equiv \exists k. \text{eval_fuel}(k, t, A) = S(r).$$

Satisfaction comes in the same two forms. For the direct evaluator, `bga_semantics` defines

```

definition sat :: "num ⇒ num ⇒ o" where
  "sat f A ≡ if tag_F f = 0
    then eval (cpx (load_F f)) A = eval (cpy (load_F f)) A
    else eval (cpx (load_F f)) A ≠ eval (cpy (load_F f)) A"

definition sat_hyp :: "hyp ⇒ asn ⇒ o" where
  "sat_hyp G A ≡ ∀f. (mem f G) → (sat f A)"

```

that is, `sat( $\varphi$ ,  $A$ )` compares `eval` of the two sides, with the comparison chosen by the formula `tag`, and `sat_hyp` quantifies over the members of the list. For the step-indexed evaluator, satisfaction of an equation also has to require both sides to converge:

```

definition sat_fuel :: "fm ⇒ asn ⇒ o" where
  "sat_fuel f A ≡ ∃x. ∃y. evals (hyp_of (load_F f)) A x ∧ (evals (conc_of (load_F f)) A y ∧
    (if tag_F f = F_EQ then x = y else x ≠ y))"

```

```

definition sat_hyp_fuel :: "hyp ⇒ asn ⇒ o" where
  "sat_hyp_fuel G A ≡ ∀f. f ∈ G → sat_fuel f A"

```

$$\text{sat_fuel}(\varphi, A) \equiv \exists x, y. \text{evals}(a, A, x) \wedge \text{evals}(b, A, y) \wedge \begin{cases} x = y & \text{tag } \varphi = \text{F_EQ} \\ x \neq y & \text{otherwise} \end{cases}$$

where a and b are the two sides of φ . A formula whose sides diverge is unsatisfied under every assignment, which is the property the consistency argument ultimately rests on.

3.9 Satisfaction at a fixed budget

`sat_fuel` is not known to be grounded. Unfolding it gives an existential over the budget, so deciding it would decide whether an evaluation converges. That matters for §4.4, where satisfaction of the hypotheses appears as the antecedent of an implication inside an induction motive, and $\rightarrow$ introduction requires the antecedent to be grounded.

The budget is the only source of the difficulty. `eval_fuel` is total, so at a fixed k every question about it is decided. A second layer of definitions pins the budget.

<code>ev_ok k t A</code>	$\triangleq \neg(\text{eval_fuel}(k, t, A) = 0)$, success within k
<code>sat_at k f A</code>	both sides succeed within k and compare as the tag requires
<code>sat_hyp_at k G A</code>	$\triangleq \forall f. f \in G \rightarrow \text{sat_at}(k, f, A)$

`sat_at` is quantifier free, so `sat_at_bool` follows from `eval_fuel_N` and `eqBool`, on the pattern of `eval_fuel_zero_bool`. `sat_hyp_at_bool` goes by list induction, and its negative case has to refute a universal, since some member of the list may fail. That is the one place in the development where $\neg\forall I$ and $\neg\forall E$ of §2.6 are used.

Both predicates are monotone in the budget by `eval_fuel_success_add`, which gives the two bridges back to §3.8.

$$\frac{\text{sat_hyp_atD} \quad \text{sat_hyp_at } k \ G \ A \implies \text{sat_hyp_fuel } G \ A}{\text{sat_hyp_at_exI} \quad \text{sat_hyp_fuel } G \ A \implies \exists k. \text{ sat_hyp_at } k \ G \ A}$$

The downward bridge is immediate. The upward one needs one budget that serves every member of the list, and `eval_fuel_success_add` supplies it by addition, the same device `eval_fuel_success_unique` uses to compare two runs.

The upward bridge has a subtlety. Proving it by list induction on G would put `sat_hyp_fuel  $G$   $A$` in the antecedent of the motive, which is the problem it exists to solve. It is therefore proved by induction over a sublist G' of a fixed G . The antecedent of the motive becomes `subset  $G'$   $G$` , which `subset_bool` decides, while `sat_hyp_fuel  $G$   $A$` stays outside the induction as an ordinary assumption. Instantiating at $G' = G$ through `subset_refl` gives the statement.

3.10 bga_subst and asn_put

```

locale bga_subst = bga_encoding +
  fixes subst_T      :: "tm  $\Rightarrow$  tm  $\Rightarrow$  tm"
  (* subst_T t j v: Inside term 't', replace variable index 'j' with term 'v' *)
  assumes subst_T_def: "subst_T t j v :=
    if tag_T t = T_VAR then
      (if load_T t = j then v else t)
    else if tag_T t = T_ZERO then (pack_T T_ZERO 0)
    else if tag_T t = T_SUC then (pack_T T_SUC (subst_T (load_T t) j v))
    else if tag_T t = T_PRED then (pack_T T_PRED (subst_T (load_T t) j v))
    else if tag_T t = T_IFZ then
      (pack_T T_IFZ (subst_T (cpx (load_T t)) j v
        , ((subst_T (cpx (cpy (load_T t))) j v),
          (subst_T (cpy (cpy (load_T t))) j v))))
    else
      (pack_T T_APP ((cpx (load_T t)), (
        (subst_T (cpx (cpy (load_T t))) j v),
        (subst_T (cpy (cpy (load_T t))) j v))))"

  fixes subst_F      :: "fm  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  fm"
  (* subst_F f j v: Inside formula 'f', replace variable index 'j' with term 'v' *)
  assumes subst_F_def: "subst_F f j v :=
    if tag_F f = F_EQ then
      pack_F F_EQ ((subst_T (cpx (load_F f)) j v), (subst_T (cpy (load_F f)) j v))
    else
      pack_F F_NEQ ((subst_T (cpx (load_F f)) j v), (subst_T (cpy (load_F f)) j v))"

  fixes subst_body :: "tm  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  tm"
  (* subst_body b x y = b[0 $\mapsto$ x, 1 $\mapsto$ y] simultaneously *)
  assumes subst_body_def: "subst_body b x y :=
    if tag_T b = T_VAR then
      (if load_T b = 0 then x else if load_T b = 1 then y else b)
    else if tag_T b = T_ZERO then b
    else if tag_T b = T_SUC then pack_T T_SUC (subst_body (load_T b) x y)
    else if tag_T b = T_PRED then pack_T T_PRED (subst_body (load_T b) x y)
    else if tag_T b = T_IFZ then
      pack_T T_IFZ (subst_body (cpx (load_T b)) x y,
        (subst_body (cpx (cpy (load_T b))) x y,
          subst_body (cpy (cpy (load_T b))) x y))

```



```

fixes asn_put :: "asn  $\Rightarrow$  num  $\Rightarrow$  val  $\Rightarrow$  asn"
assumes asn_put_def: "asn_put A i v :=
  if i = 0 then
    if A = Nil then v  $\triangleright$  Nil else v  $\triangleright$  list_tl A
  else if A = Nil then 0  $\triangleright$  asn_put Nil (i - 1) v
  else list_hd A  $\triangleright$  asn_put (list_tl A) (i - 1) v"
```

$\text{subst_T} :: \text{tm} \Rightarrow \text{tm} \Rightarrow \text{tm} \Rightarrow \text{tm}$ $\text{subst_F} :: \text{fm} \Rightarrow \text{tm} \Rightarrow \text{tm} \Rightarrow \text{fm}$ $\text{subst_body} :: \text{tm} \Rightarrow \text{tm} \Rightarrow \text{tm} \Rightarrow \text{tm}$ $\text{asn_put} :: \text{asn} \Rightarrow \text{num} \Rightarrow \text{val} \Rightarrow \text{asn}$	$t[v_j \mapsto v]$, replace variable j inside a term the same inside a formula, side by side $b[v_0 \mapsto x, v_1 \mapsto y]$, simultaneous $A[i \mapsto v]$, update an assignment
---	---

All three substitutions recurse over the tag and rebuild with `pack_T`. Two details are worth noting. In the `T_ZERO` case `subst_T` returns `pack_T T_ZERO 0` rather than the original code, which normalizes the ignored payload of a zero. In the `T_APP` case the definition index `cpx (load_T t)` is carried through untouched, since it is not a term. `subst_body` is the two-variable version used by the application rule, and it exists separately because that rule substitutes both arguments at once. `asn_put A i v` walks down i entries and replaces the one it finds, padding with zeros when i runs past the end of the list, so that an assignment is a total map from indices to values.

3.11 The proof rules of BGA

The rules are given below in sequent form, and each is realized as one branch of the encoded checker. Here Γ is a list of encoded formulas, φ an encoded formula, and $\varphi[a]$ a formula with occurrences of a marked, which is realized by the template search of §3.12. The turnstile is the BGA sequent arrow, written $\Vdash$ in the sources.

$$\begin{array}{c}
\frac{}{\Gamma, \varphi \vdash \varphi} \text{ (HYP)} \\
\frac{\Gamma \vdash a = b}{\Gamma \vdash b = a} \text{ (=S)} \\
\frac{\Gamma \vdash a = b}{\Gamma \vdash S(a) = S(b)} \text{ (S=I)} \\
\frac{\Gamma \vdash a \text{ N}}{\Gamma \vdash P(S(a)) = a} \text{ (PI)} \\
\frac{\Gamma \vdash c = 0 \quad \Gamma \vdash a \text{ N}}{\Gamma \vdash (c \text{ 0? } a : b) = a} \text{ (?I1)} \\
\frac{\Gamma \vdash a \neq b}{\Gamma \vdash b \neq a} \text{ ($\neq$ S)} \\
\frac{\Gamma \vdash S(a) \neq S(b)}{\Gamma \vdash a \neq b} \text{ (S $\neq$ E)} \\
\frac{\Gamma' \vdash \varphi \quad \Gamma' \subseteq \Gamma}{\Gamma \vdash \varphi} \text{ (wk)} \\
\frac{\Gamma \vdash a_0 \text{ N} \quad \Gamma \vdash a_1 \text{ N} \quad \Gamma \vdash \varphi[D_a\langle a_0, a_1 \rangle]}{\Gamma \vdash \varphi[d(a_0, a_1)]} \text{ (APP)}
\end{array}
\qquad
\begin{array}{c}
\frac{}{\Gamma \vdash 0 = 0} \text{ (OI)} \\
\frac{\Gamma \vdash a = b \quad \Gamma \vdash \varphi[a]}{\Gamma \vdash \varphi[b]} \text{ (=E)} \\
\frac{\Gamma \vdash S(a) = S(b)}{\Gamma \vdash a = b} \text{ (S=E)} \\
\frac{\Gamma \vdash a \text{ N}}{\Gamma \vdash S(a) \neq 0} \text{ (S $\neq$ OI)} \\
\frac{\Gamma \vdash c \neq 0 \quad \Gamma \vdash b \text{ N}}{\Gamma \vdash (c \text{ 0? } a : b) = b} \text{ (?I2)} \\
\frac{\Gamma \vdash a \neq b}{\Gamma \vdash S(a) \neq S(b)} \text{ ($\neq$ SI)} \\
\frac{\Gamma \vdash a \quad \Gamma, a \vdash c}{\Gamma \vdash c} \text{ (CUT)} \\
\frac{\Gamma \vdash a \text{ N} \quad \Gamma \vdash \varphi[0] \quad \Gamma, v_i \text{ N}, \varphi[v_i] \vdash \varphi[S(v_i)]}{\Gamma \vdash \varphi[a]} \text{ (IND)}
\end{array}$$

Termination is not a separate judgment form in BGA. $a \text{ N}$ abbreviates the formula $a = a$. Almost every rule is an introduction rule, and the only elimination is $=E$.

The checker never applies a rule forwards. It is given a judgment J and the tail ρ of the proof, and asks whether J could have been produced by some rule from judgments occurring in ρ . Every rule with a premise therefore becomes a search through ρ , and the rules whose premises mention a $\varphi[\cdot]$ become a search through ρ combined with a search for a template.

3.12 bga_subst_rule

<code>rep_vars_T :: tm $\Rightarrow$ tm $\Rightarrow$ tm</code>	replaces every leaf of a term by a given variable
<code>rep_vars_F :: fm $\Rightarrow$ tm $\Rightarrow$ fm</code>	the same on both sides of a formula
<code>check_template</code>	searches for a template below a bound
<code>find_phi</code>	searches ρ for the premise $\Gamma \vdash \varphi[a]$
<code>find_eq</code>	searches ρ for the equation $\Gamma \vdash a = b$
<code>check_subst :: jdg $\Rightarrow$ pf $\Rightarrow$ o</code>	the rule $=E$ as a whole

The substitution rule is the one place where the checker has to invent something. To justify $\Gamma \vdash \varphi[b]$ it must produce three things: a premise $\Gamma \vdash \varphi[a]$ somewhere in ρ , an equation $\Gamma \vdash a = b$ somewhere in ρ , and a *template* p , a formula with a designated variable v_i , such that

$$p[v_i \mapsto a] = \varphi[a] \quad \text{and} \quad p[v_i \mapsto b] = \varphi[b].$$

The template records which occurrences of a are being rewritten. Nothing in the conclusion says what it is, so it is searched for.

Choosing the variable. The designated variable must not already occur in the judgment, or substituting into the template would disturb something else. The index chosen is $J + 1$, where J is the code of the whole judgment. This is fresh because a variable v_j is encoded as $\text{pack}_T(\text{T_VAR}, j)$, which is a pair with second component j , and pairing and packing are monotone in that component, so $j \leq J$ for every variable occurring anywhere in J . Monotonicity of pack in §3.6 is assumed for this argument.

Bounding the search. Templates are enumerated downwards from $\text{rep_vars}_F(\varphi[b], J + 1)$, the formula obtained by replacing every leaf of the conclusion by v_{J+1} :

$$\text{rep_vars}_T(t, i) = \begin{cases} v_i & t = v_j \text{ or } t = 0 \\ S(\text{rep_vars}_T(a, i)) & t = S(a) \\ P(\text{rep_vars}_T(a, i)) & t = P(a) \\ (\text{rep_vars}_T(c, i) \text{ 0? } \text{rep_vars}_T(a, i) : \text{rep_vars}_T(b, i)) & t = (c \text{ 0? } a : b) \\ d_j(\text{rep_vars}_T(a, i), \text{rep_vars}_T(b, i)) & t = d_j(a, b) \end{cases}$$

This is the template that abstracts *every* leaf at once. Its code bounds the code of every other candidate with the same constructor skeleton, since such a candidate differs from it only by leaving some leaves unabstracted, and a leaf is numerically smaller than v_{J+1} . `check_template` then counts p down from that bound to zero and tests each value:

$$\text{check_template}(f, \varphi, a, b, p, i) \iff \exists q \leq p. q[v_i \mapsto a] = \varphi \wedge q[v_i \mapsto b] = f.$$

```
fixes check_template :: "fm  $\Rightarrow$  fm  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  fm  $\Rightarrow$  tm  $\Rightarrow$  o"
(* Counts down p from max_bound to 0.
   i is fixed to a fresh variable (e.g., f+1)
```

```

f - formula we are trying to see if valid
phi - premise formula
p - template formula
i - id of variable acting as a place holder in p
a=b - equality being considered
*)
assumes check_template_def: "check_template f phi a b p i :=
  if subst_F p i a = phi ∧ subst_F p i b = f then True
  else if p > 0 = 1 then check_template f phi a b (p - 1) i
  else False"

```

```

fixes rep_vars_T      :: "tm ⇒ tm ⇒ tm"
(* rep_vars_T t i: Replaces all leaves in term t with variable i *)
assumes rep_vars_T_def: "rep_vars_T t i :=
  if tag_T t = T_VAR then pack_T T_VAR i
  else if tag_T t = T_ZERO then pack_T T_VAR i
  else if tag_T t = T_SUC then pack_T T_SUC (rep_vars_T (load_T t) i)
  else if tag_T t = T_PRED then pack_T T_PRED (rep_vars_T (load_T t) i)
  else if tag_T t = T_IFZ then
    pack_T T_IFZ (rep_vars_T (cpx (load_T t)) i,
      (rep_vars_T (cpx (cpy (load_T t))) i,
        rep_vars_T (cpy (cpy (load_T t))) i))

```

Remark 3.2 (The bound matters for completeness only). Soundness of the checker does not depend on the bound being correct. `check_template_sound_fuel` assumes that the search reported success and extracts the witness q it found; a bound that was too small would make the checker reject proofs it ought to accept, which is a completeness failure, and would leave soundness untouched. Nothing in this development proves completeness.

Searching the proof. With the template search in place, the two remaining searches walk the proof list. In mathematical notation, writing $K \in \rho$ for membership in the proof list,

$$\begin{aligned}
\text{find_phi}(J, a, b, \pi) &\iff \exists K \in \pi. \text{hyp}(K) = \text{hyp}(J) \wedge \text{check_template}(\text{conc}(J), \text{conc}(K), a, b, \beta_J, J + 1), \\
\text{find_eq}(J, \rho, \pi) &\iff \exists K \in \pi. \text{hyp}(K) = \text{hyp}(J) \wedge \text{conc}(K) \text{ is } (a = b) \wedge \text{find_phi}(J, a, b, \rho), \\
\text{check_subst}(J, \rho) &\iff \text{find_eq}(J, \rho, \rho),
\end{aligned}$$

where $\beta_J = \text{rep_vars}_F(\text{conc}(J), J + 1)$ is the template bound. Both searches carry two proof arguments. The one named `ptr` is the part still to be scanned and shrinks on each step, and the one named `rest` is the whole tail of the proof and stays fixed, because the premise being looked for may sit anywhere in it. The hypothesis test $\text{hyp}(K) = \text{hyp}(J)$ is what keeps the rule from silently changing the context.

```

fixes find_phi      :: "jdg ⇒ tm ⇒ tm ⇒ pf ⇒ o"
(* Scans the proof list (ptr) for a valid premise J_phi = ⟨Γ, φ⟩
  such that Γ matches J's context, and p[v-(f+1) ↦ a] = φ and p[v-(f+1) ↦ b] = f *)
assumes find_phi_def: "find_phi J a b ptr :=
  if ptr = Nil then False
  else if hyp_of (list_hd ptr) = hyp_of J ∧
    check_template (conc_of J) (conc_of (list_hd ptr)) a b
      (rep_vars_F (conc_of J) (J + 1)) (J + 1) then True
  else find_phi J a b (list_tl ptr)"

```

```

fixes find_eq      :: "jdg ⇒ pf ⇒ pf ⇒ o"
(* Scans the proof list (ptr) for an equality judgment J_eq = ⟨Γ, a = b⟩
  such that Γ matches J's context, and calls find_phi *)
assumes find_eq_def: "find_eq J rest ptr :=

```

```

if ptr = Nil then False
else if hyp_of (list_hd ptr) = hyp_of J  $\wedge$  tag_F (conc_of (list_hd ptr)) = F_EQ then
  if find_phi J (cpx (load_F (conc_of (list_hd ptr)))) (cpy (load_F (conc_of (list_hd
    ptr)))) rest then True
  else find_eq J rest (list_tl ptr)
else find_eq J rest (list_tl ptr)"
fixes check_subst    :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  o"

(* checking the substitution rule *)
assumes check_subst_def: "check_subst J rest := find_eq J rest rest"

```

3.13 bga_eq_rule

check_eq_rules :: hyp $\Rightarrow$ tm $\Rightarrow$ tm $\Rightarrow$ tmtag $\Rightarrow$ tmtag $\Rightarrow$ pf $\Rightarrow$ o	the seven rules concluding an equation
check_neq_rules :: hyp $\Rightarrow$ tm $\Rightarrow$ tm $\Rightarrow$ tmtag $\Rightarrow$ tmtag $\Rightarrow$ pf $\Rightarrow$ o	the four rules concluding a disequation

Both take the two sides and their tags as separate arguments, already projected out of the conclusion by valid_step, so that the branches can test tags without recomputing them.

```

fixes check_eq_rules  :: "hyp  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  tmtag  $\Rightarrow$  tmtag  $\Rightarrow$  pf  $\Rightarrow$  o"
(* Equality Rules
1.  $\Gamma \vdash 0 = 0$ 
2.  $\Gamma \vdash a = b \Rightarrow \Gamma \vdash b = a$ 
3.  $\Gamma \vdash a = b \Rightarrow \Gamma \vdash S(a) = S(b)$ 
4.  $\Gamma \vdash S(a) = S(b) \Rightarrow \Gamma \vdash a = b$ 
5.  $\Gamma \vdash a = 0 \Rightarrow \Gamma \vdash P(S(a)) = a$ 
6.  $\Gamma \vdash c \neq 0 \Rightarrow \Gamma \vdash b = 0 \Rightarrow \Gamma \vdash (c = 0 ? a : b) = b$ 
7.  $\Gamma \vdash c = 0 \Rightarrow \Gamma \vdash a = 0 \Rightarrow \Gamma \vdash (c = 0 ? a : b) = a$ 
*)
assumes check_eq_rules_def: "check_eq_rules G lhs rhs tg_L tg_R rest :=
  if lhs = pack_T T_ZERO 0  $\wedge$  rhs = pack_T T_ZERO 0 then True
  else if mem (G  $\Vdash$  pack_F F_EQ (rhs, lhs)) rest then True
  else if tg_L = T_SUC  $\wedge$  tg_R = T_SUC  $\wedge$ 
    mem (G  $\Vdash$  pack_F F_EQ (load_T lhs, load_T rhs)) rest then True
  else if mem (G  $\Vdash$  pack_F F_EQ (pack_T T_SUC lhs, pack_T T_SUC rhs)) rest then True
  else if tg_L = T_PRED  $\wedge$  tag_T (load_T lhs) = T_SUC  $\wedge$ 
    (load_T (load_T lhs)) = rhs  $\wedge$ 
    mem (G  $\Vdash$  pack_F F_EQ (rhs, rhs)) rest then True
  else if tg_L = T_IFZ  $\wedge$  rhs = cpy (cpy (load_T lhs))  $\wedge$ 
    mem (G  $\Vdash$  pack_F F_NEQ (cpx (load_T lhs), pack_T T_ZERO 0)) rest  $\wedge$ 
    mem (G  $\Vdash$  pack_F F_EQ (rhs, rhs)) rest then True
  else if tg_L = T_IFZ  $\wedge$  rhs = cpx (cpy (load_T lhs))  $\wedge$ 
    mem (G  $\Vdash$  pack_F F_EQ (cpx (load_T lhs), pack_T T_ZERO 0)) rest  $\wedge$ 
    mem (G  $\Vdash$  pack_F F_EQ (rhs, rhs)) rest then True
  else False"

```

The seven branches are, in order, 0I, =S, S=I, S=E, PI, ?I2 and ?I1. Each is a membership test on rest, so a judgment is justified only by judgments occurring later in the list, which is what makes check_list a well-founded recursion. Reading the fifth branch as an example: the conclusion is $\Gamma \vdash P(u) = b$, the branch checks that u is $S(b)$ by inspecting tags and payloads, and it requires $\Gamma \vdash b = b$ to appear in the proof, which is the habeas quid premise b N of PI.

```

fixes check_neq_rules :: "hyp  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  tmtag  $\Rightarrow$  tmtag  $\Rightarrow$  pf  $\Rightarrow$  o"
(* Not-Equality Rules
1.  $\Gamma \vdash a \neq b \Rightarrow \Gamma \vdash b \neq a$ 
2.  $\Gamma \vdash a = 0 \Rightarrow \Gamma \vdash S(a) \neq 0$ 
3.  $\Gamma \vdash a \neq b \Rightarrow \Gamma \vdash S(a) \neq S(b)$ 

```

```

4.  $\Gamma \vdash S(a) \neq S(b) \implies \Gamma \vdash a \neq b$ 
*)
assumes check_neq_rules_def: "check_neq_rules G lhs rhs tg_L tg_R rest :=
  if mem (G  $\Vdash$  pack_F F_NEQ (rhs, lhs)) rest then True
  else if tg_L = T_SUC  $\wedge$  rhs = pack_T T_ZERO 0  $\wedge$ 
    mem (G  $\Vdash$  pack_F F_EQ (load_T lhs, load_T lhs)) rest then True
  else if tg_L = T_SUC  $\wedge$  tg_R = T_SUC  $\wedge$ 
    mem (G  $\Vdash$  pack_F F_NEQ (load_T lhs, load_T rhs)) rest then True
  else if mem (G  $\Vdash$  pack_F F_NEQ (pack_T T_SUC lhs, pack_T T_SUC rhs)) rest then True
  else False"

```

and dually $\neq S, S \neq 0I, \neq SI$ and $S \neq E$.

3.14 bga_ind_rule

check_ind_template	tests one candidate template for the induction rule
find_ind_base	searches ρ for the base case
check_ind :: jdg $\Rightarrow$ pf $\Rightarrow$ o	the rule IND as a whole

Induction needs the same kind of template, and three premises rather than one. To justify $\Gamma \vdash \varphi[a]$ it looks for a template q with

$q[v_i \mapsto a] = \varphi[a]$, $q[v_i \mapsto 0]$ present in ρ , $(v_i = v_i, q, \Gamma \vdash q[v_i \mapsto S(v_i)])$ present in ρ ,

that is,

$$\text{check_ind_template}(f, \varphi, a, \Gamma, p, i, \rho) \iff \exists q \leq p. q[v_i \mapsto a] = f \wedge q[v_i \mapsto 0] = \varphi \wedge (v_i = v_i, q, \Gamma \vdash q[v_i \mapsto S(v_i)]) \in \rho.$$

```

fixes check_ind_template :: "fm  $\Rightarrow$  fm  $\Rightarrow$  tm  $\Rightarrow$  hyp  $\Rightarrow$  fm  $\Rightarrow$  tm  $\Rightarrow$  pf  $\Rightarrow$  o"
(* counts down 'p', testing if it is the valid induction template
checks for a p such that:
p[v_i  $\mapsto$  a] =  $\varphi$ 
 $\Gamma \vdash p[v_i \mapsto 0]$  in proof
[v_i = v_i, p] +  $\Gamma \vdash p[v_i \mapsto S(v_i)]$  in proof
*)
assumes check_ind_template_def: "check_ind_template f phi a G p i rest :=
  if subst_F p i a = f  $\wedge$ 
    subst_F p i (pack_T T_ZERO 0) = phi  $\wedge$ 
    mem (pack_F F_EQ (pack_T T_VAR i, pack_T T_VAR i)  $\triangleright$  p  $\triangleright$  G  $\Vdash$ 
      subst_F p i (pack_T T_SUC (pack_T T_VAR i))) rest
  then True
  else if p > 0 = 1 then check_ind_template f phi a G (p - 1) i rest
  else False"

```

```

fixes check_ind :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  o"
(* check_ind triggers the search if the Habeas Quid premise  $\Gamma \vdash a = a$  exists
 $\hookrightarrow$  *)
assumes check_ind_def: "check_ind J rest :=
  if fresh_H (J + 1) (hyp_of J)  $\wedge$  mem (hyp_of J  $\Vdash$  pack_F F_EQ (cpx (load_F (conc_of J)), cpx
     $\hookrightarrow$  (load_F (conc_of J)))) rest then
    find_ind_base J (cpx (load_F (conc_of J))) rest rest
  else False"

```

check_ind guards the search with two conditions. The habeas quid premise $\Gamma \vdash a = a$, encoded as the judgment $\Gamma \vdash a = a$, must be in the proof, and $\text{fresh}_H(J + 1, \Gamma)$ must hold, so that the

variable used to build the template does not occur in the hypotheses. The step case sought in the proof carries both the induction hypothesis q and the termination assumption $v_i = v_i$ on the induction variable, which is what the semantic induction lemma of §4 consumes. `find_ind_base` scans ρ for the base case in the same style as `find_phi`.

3.15 bga_struct_rule

<code>find_cut</code>	searches ρ for the cut formula
<code>check_cut :: jdg $\Rightarrow$ pf $\Rightarrow$ o</code>	the rule <code>CUT</code>
<code>find_struct</code>	searches ρ for the weakened judgment
<code>check_struct :: jdg $\Rightarrow$ pf $\Rightarrow$ o</code>	the rule <code>WK</code>

```

locale bga_struct_rule = bga_encoding +

fixes find_cut      :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  pf  $\Rightarrow$  o"
fixes check_cut     :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  o"

(* Cut Rule:
 $\Gamma \vdash a \Rightarrow a + \Gamma \vdash c \Rightarrow \Gamma \vdash c$ 
*)
assumes find_cut_def: "find_cut J rest ptr :=
  if ptr = Nil then False
  else if hyp_of (list_hd ptr) = hyp_of J then
    if mem ((conc_of (list_hd ptr))  $\triangleright$  (hyp_of J)  $\Vdash$  (conc_of J)) rest then True
    else find_cut J rest (list_tl ptr)
  else find_cut J rest (list_tl ptr)"

assumes check_cut_def: "check_cut J rest := find_cut J rest rest"

fixes find_struct :: "jdg  $\Rightarrow$  hyp  $\Rightarrow$  pf  $\Rightarrow$  o"
assumes find_struct_def: "find_struct J G ptr :=
  if ptr = Nil then False
  else if conc_of (list_hd ptr) = conc_of J  $\wedge$  subset (hyp_of (list_hd ptr)) G then True
  else find_struct J G (list_tl ptr)"

fixes check_struct :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  o"
assumes check_struct_def: "check_struct J rest := find_struct J (hyp_of J) rest"

```

$$\text{check_cut}(J, \rho) \iff \exists K \in \rho. \text{hyp}(K) = \text{hyp}(J) \wedge (\text{conc}(K), \text{hyp}(J) \vdash \text{conc}(J)) \in \rho,$$

$$\text{check_struct}(J, \rho) \iff \exists K \in \rho. \text{conc}(K) = \text{conc}(J) \wedge \text{hyp}(K) \subseteq \text{hyp}(J).$$

The hypothesis rule is not a separate function. It is the first branch of `valid_step`, the test `mem (conc_of J) (hyp_of J)`.

3.16 bga_app_rule

<code>app_try</code>	tests one triple (d, x, y)
<code>app_y</code>	counts y down
<code>app_x</code>	counts x down
<code>app_d</code>	counts the definition index d down
<code>check_app :: jdg $\Rightarrow$ pf $\Rightarrow$ o</code>	the rule <code>APP</code> as a whole

```

fixes app_try :: "jdg  $\Rightarrow$  num  $\Rightarrow$  num  $\Rightarrow$  num  $\Rightarrow$  pf  $\Rightarrow$  o"
assumes app_try_def: "app_try J d x y rest :=
  dfn_is d 2 (nth d dfns)  $\wedge$ 
  mem (hyp_of J  $\Vdash$  pack_F F_EQ (x, x)) rest  $\wedge$ 
  mem (hyp_of J  $\Vdash$  pack_F F_EQ (y, y)) rest  $\wedge$ 
  find_phi J (subst_body (nth d dfns) x y) (pack_T T_APP (d, (x, y))) rest"

(* Tries to see if application is satisfied for some y*)
fixes app_y :: "jdg  $\Rightarrow$  num  $\Rightarrow$  num  $\Rightarrow$  num  $\Rightarrow$  pf  $\Rightarrow$  o"
assumes app_y_def: "app_y J d x y rest :=
  if app_try J d x y rest then True
  else if y > 0 = 1 then app_y J d x (y - 1) rest
  else False"

(* Tries to see if application is satisfied for some x, y with y bounded by f (where J=  $\Gamma \vdash f$ ) *)
fixes app_x :: "jdg  $\Rightarrow$  num  $\Rightarrow$  num  $\Rightarrow$  pf  $\Rightarrow$  o"
assumes app_x_def: "app_x J d x rest :=
  if app_y J d x (conc_of J) rest then True
  else if x > 0 = 1 then app_x J d (x - 1) rest
  else False"

(*Given a J and an upperbound for def, sees if application can be satisfied by any d,x,y
   triplet *)
fixes app_d :: "jdg  $\Rightarrow$  num  $\Rightarrow$  pf  $\Rightarrow$  o"
assumes app_d_def: "app_d J d rest :=
  if d < len dfns = 1 then
    (if app_x J d (conc_of J) rest then True
     else if d > 0 = 1 then app_d J (d - 1) rest else False)
  else
    (if d > 0 = 1 then app_d J (d - 1) rest else False)"

fixes check_app :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  o"
assumes check_app_def: "check_app J rest := app_d J (len dfns - 1) rest"

```

Application unfolds a definition. To justify $\Gamma \vdash \varphi[d(x,y)]$ the checker needs $\Gamma \vdash x \mathbf{N}$, $\Gamma \vdash y \mathbf{N}$ and a premise $\Gamma \vdash \varphi[D_d[v_0 \mapsto x, v_1 \mapsto y]]$:

$$\text{app_try}(J, d, x, y, \rho) \iff \text{dfn_is}(d, 2, D_d) \wedge (\text{hyp}(J) \vdash x = x) \in \rho \wedge (\text{hyp}(J) \vdash y = y) \in \rho \\ \wedge \text{find_phi}(J, D_d[v_0 \mapsto x, v_1 \mapsto y], d(x, y), \rho).$$

Note that `find_phi` is reused here with the two sides of the rewrite being the unfolded body and the application, so unfolding a definition is treated as one more instance of rewriting inside a template.

None of d , x and y is determined by the conclusion, so all three are enumerated:

$$\text{check_app}(J, \rho) \iff \exists d < |D|. \exists x \leq \text{conc}(J). \exists y \leq \text{conc}(J). \text{app_try}(J, d, x, y, \rho).$$

The three nested countdowns `app_d`, `app_x` and `app_y` are that triple search written as recursion, since QGA has no bounded quantifier. The bound $\text{conc}(J)$ on the arguments is the same kind of bound as in §3.12: the code of a subterm is at most the code of the formula containing it, and as before it is completeness rather than soundness that depends on it.

3.17 bga_proof_check

<code>valid_step :: jdg $\Rightarrow$ pf $\Rightarrow$ o</code>	<i>"J follows by one rule from judgments in ρ"</i>
<code>check_list :: pf $\Rightarrow$ o</code>	<i>every entry follows from the entries after it</i>
<code>is_valid_proof :: pf $\Rightarrow$ jdg $\Rightarrow$ o</code>	<i>"π is a valid proof whose head is J"</i>

```

fixes valid_step :: "jdg ⇒ pf ⇒ o"

(* The main step check
1. 1 hyp Rule
2. 1 Cut Rule
3. 1 Subst Rule
4. 1 Induction Rule
5. 7 Equality Rules
6. 4 Not-equality rules
7. 1 app2I rule
8. 1 wk1 rule
sub1
*)
assumes valid_step_def: "valid_step J rest :=
  if mem (conc_of J) (hyp_of J) then True
  else if check_cut J rest then True
  else if check_subst J rest then True
  else if check_ind J rest then True
  else if check_app J rest then True
  else if check_struct J rest then True
  else if tag_F (conc_of J) = F_EQ then
    check_eq_rules (hyp_of J) (cpx (load_F (conc_of J))) (cpy (load_F (conc_of J)))
      (tag_T (cpx (load_F (conc_of J)))) (tag_T (cpy (load_F (conc_of J)))) rest
  else
    check_neq_rules (hyp_of J) (cpx (load_F (conc_of J))) (cpy (load_F (conc_of J)))
      (tag_T (cpx (load_F (conc_of J)))) (tag_T (cpy (load_F (conc_of J)))) rest"

```

`valid_step` is the disjunction of the branches, tried in order: membership for `hyp`, then cut, substitution, induction, application and weakening, and finally the equality or disequality rules according to the tag of the conclusion. Written as a formula,

$$\begin{aligned}
\text{valid_step}(J, \rho) \iff & \text{conc}(J) \in \text{hyp}(J) \vee \text{check_cut}(J, \rho) \vee \text{check_subst}(J, \rho) \\
& \vee \text{check_ind}(J, \rho) \vee \text{check_app}(J, \rho) \vee \text{check_struct}(J, \rho) \\
& \vee (\text{tag conc}(J) = \text{F_EQ} ? \text{check_eq_rules}(\dots) : \text{check_neq_rules}(\dots)).
\end{aligned}$$

The order is not significant for soundness. It is a sequence of nested conditionals rather than a disjunction because each branch has to be evaluated only when the earlier ones fail.

```

fixes check_list :: "pf ⇒ o"
assumes check_list_def: "check_list pf :=
  if pf = Nil then True
  else if valid_step (list_hd pf) (list_tl pf) then check_list (list_tl pf)
  else False"

```

```

assumes is_valid_proof_def: "is_valid_proof pf J :=
  if pf = Nil then False
  else if list_hd pf = J then check_list pf
  else False"

```

A proof is valid when every entry follows from the entries after it, and it proves J when additionally its head is J :

$$\text{check_list}(\pi) \iff \pi = \text{Nil} \vee (\text{valid_step}(\text{hd } \pi, \text{tl } \pi) \wedge \text{check_list}(\text{tl } \pi)),$$

$$\text{is_valid_proof}(\pi, J) \iff \pi \neq \text{Nil} \wedge \text{hd } \pi = J \wedge \text{check_list}(\pi).$$

Reading the list head first, with each entry justified by its tail, is what makes the recursion terminate and what the soundness induction of §4.4 follows.

4 The consistency proof

This section follows the argument from the evaluator up to the final theorem. The intent is to make the shape of the proof and its load-bearing lemmas visible; the detailed proofs are left to a later instalment.

4.1 Shape of the argument

Everything reduces to one implication, called the soundness bridge:

if p checks as a proof of J , and the hypotheses of J are satisfied by an assignment A , then the conclusion of J is satisfied by A .

Given it, §3.4 closes the argument in half a page. Proving it requires four things, and they are the subsections that follow.

1. The step-indexed evaluator determines at most one value, so that `evals` behaves as a partial function and satisfaction is well behaved.
2. Every branch of `valid_step` preserves satisfaction, assuming that every judgment in the tail of the proof does.
3. That local statement can be lifted along the proof list by induction.
4. Satisfaction of an encoded equation and of an encoded disequality can be unpacked into statements about values.

A remark on technique before the details. QGA induction instantiates a schematic predicate Q , so a statement can only be generalized before inducting if the generalization is part of the object-level formula. Several proofs in this file are therefore stated internally in the form $\forall u. \forall A2. \dots$, or as nested implications $\dots \rightarrow (\dots \rightarrow \dots)$, and unpacked afterwards with `forallE` and `implE`. `eval_fuel_N` is the first instance: the induction on `fuel` runs on $\forall u. \forall A2. \text{eval_fuel } k \ u \ A2 \ N$ and the desired statement is recovered by two instantiations.

Nesting an implication inside a motive has a price. $\rightarrow$ introduction requires its antecedent to be grounded, so every antecedent of such a motive needs a groundedness lemma. The checkers all have one. Satisfaction does not, and §3.9 is what supplies it.

4.2 The evaluator is a partial function

The fuel evaluator is total as a function into `num`, which is the statement `eval_fuel_N:  $\llbracket k \ N; \ t \ N; \ A \ N \rrbracket \Rightarrow \text{eval\_fuel } k \ t \ A \ N$` , and 0 is its “no result” value. Success is preserved when the budget grows.

```
lemma eval_fuel_success_add:
  assumes k: "k N" and n: "n N" and t: "t N" and A: "A N" and result: "eval_fuel k t A = S r"
  shows "eval_fuel (k + n) t A = S r"
```

This is proved by induction on the increment, from the one-step version `eval_fuel_success_suc`. Two successful runs with different budgets agree, since both can be raised to the common budget $k + l$.

```
lemma eval_fuel_success_unique:
  assumes k: "k N" and l: "l N" and t: "t N" and A: "A N"
  and left: "eval_fuel k t A = S r" and right: "eval_fuel l t A = S q"
  shows "r = q"
```

```
shows "r = q"
```

The existential form follows, and it is the assumption `evals_det` that the abstract locale of §3.4 demanded.

```
lemma evals_functional:
  assumes t: "t N" and A: "A N" and left: "evals t A r" and right: "evals t A q"
  shows "r = q"
```

The remaining facts about the evaluator are one lemma per constructor, in two directions each: from a successful run one can read off the value of the subterms (`eval_fuel_suc_value`, `eval_fuel_pred_value`, `eval_fuel_ifz_zero_value`, `eval_fuel_app_value`), and from successful runs of the subterms one can build a successful run of the whole (`eval_fuel_sucI` and its relatives). Together they let `evals` be used as an inductively defined big-step relation, which is how the rule soundness proofs consume it.

4.3 Soundness of a single step

The central local statement is the following. It says that a checked step is sound relative to the tail of the proof, and it is what the induction of §4.4 will feed itself.

```
lemma valid_step_sound_fuel:
  assumes J: "J N" and rest: "rest N" and A: "A N"
  and vs: "valid_step J rest"
  and prev:
    " $\wedge K A2. A2 N \implies K N \implies \text{mem } K \text{ rest} \implies$ 
     $\text{sat\_hyp\_fuel } (\text{hyp\_of } K) A2 \implies$ 
     $\text{sat\_fuel } (\text{conc\_of } K) A2$ "
  and satG: "sat_hyp_fuel (hyp_of J) A"
  shows "sat_fuel (conc_of J) A"
```

Its proof is a case analysis over the branches of `valid_step`, using `cases_bool` on each guard in turn, with one lemma per branch. All the branch lemmas have the same premises as above and the same conclusion; they differ only in which checker they assume.

Branch	Lemma
hypothesis	<code>inline</code> , from <code>sat_hyp_fuel_mem</code>
cut	<code>check_cut_sound_fuel</code>
substitution of equals	<code>check_subst_sound_fuel</code>
induction	<code>check_ind_sound_fuel</code>
application	<code>check_app_sound_fuel</code>
weakening	<code>check_struct_sound_fuel</code>
the seven equality rules	<code>check_eq_rules_sound_fuel</code>
the four disequality rules	<code>check_neq_rules_sound_fuel</code>

Two of these carry the real content.

The template lemma. Substitution of equals, application and induction all reduce to the same semantic fact: if a template `p` instantiated at `a` is satisfied, and `a` and `b` evaluate to a common value, then the same template instantiated at `b` is satisfied.

```

lemma template_instance_sound_fuel:
  assumes p: "p N" and i: "i N" and a: "a N" and b: "b N"
    and phi: "phi N" and f: "f N" and A: "A N" and v: "v N"
    and pa: "subst_F p i a = phi"
    and pb: "subst_F p i b = f"
    and sph: "sat_fuel phi A"
    and eva: "evals a A v"
    and evb: "evals b A v"
  shows "sat_fuel f A"

```

It rests on a pair of substitution lemmas that relate syntactic substitution to updating the assignment.

```

lemma sat_fuel_subst_FD:
  assumes f: "f N" and i: "i N" and s: "s N" and A: "A N" and v: "v N"
    and evs: "evals s A v" and sat: "sat_fuel (subst_F f i s) A"
  shows "sat_fuel f (asn_put A i v)"

```

```

lemma sat_fuel_subst_FI:
  assumes f: "f N" and i: "i N" and s: "s N" and A: "A N" and v: "v N"
    and evs: "evals s A v" and sat: "sat_fuel f (asn_put A i v)"
  shows "sat_fuel (subst_F f i s) A"

```

Given a and b that both evaluate to v , the first pushes satisfaction of $p[i \mapsto a]$ under A down to satisfaction of p under $\text{asn_put } A \ i \ v$, and the second lifts it back up to $p[i \mapsto b]$. The search functions are then sound because they report success only when such a template exists, which is the content of `check_template_sound_fuel` and of

```

lemma find_phi_sound_fuel:
  assumes J: "J N" and a: "a N" and b: "b N"
    and rest: "rest N" and ptr: "ptr N" and A: "A N"
    and sub: "subset ptr rest"
    and fp: "find_phi J a b ptr"
    and prev: " $\wedge K. K \text{ N} \Rightarrow \text{mem } K \text{ rest} \Rightarrow$ 
      sat_hyp_fuel (hyp_of K) A  $\Rightarrow$ 
      sat_fuel (conc_of K) A"
    and satG: "sat_hyp_fuel (hyp_of J) A"
    and tr: " $\wedge q. q \text{ N} \Rightarrow \text{evals } a \ A \ q \Rightarrow \text{evals } b \ A \ q$ "
  shows "sat_fuel (conc_of J) A"

```

The premise `prev` is the induction hypothesis of §4.4 in the form the search needs: every judgment occurring in `rest` is sound. The premise `sub` records that the pointer walks a sublist of `rest`, which is what keeps the recursion inside the part of the proof already known to be sound. Application is the same lemma applied to the template $\varphi[\text{body}_d[v_0 \mapsto x, v_1 \mapsto y]]$, with the two habeas quid premises supplying the values that `subst_body` needs.

The induction rule. Soundness of the induction rule is the one place where a semantic induction is performed, over the value of the term the rule inducts on.

```

lemma nat_ind_sound_put_fuel:
  assumes p: "p N" and i: "i N" and a: "a N"
    and G: "G N" and A: "A N" and n: "n N"
    and fresh: "fresh_H i G"
    and satG: "sat_hyp_fuel G A"
    and base: "sat_fuel (subst_F p i (pack_T T_ZERO 0)) A"
    and step:

```

```

"Λm. m N ⇒
  sat_hyp_fuel
    (pack_F F_EQ
      (pack_T T_VAR i, pack_T T_VAR i)
      ▷ p ▷ G)
  (asn_put A i m) ⇒
  sat_fuel
    (subst_F p i
      (pack_T T_SUC (pack_T T_VAR i)))
  (asn_put A i m)"
and eva: "evals a A n"
shows "sat_fuel (subst_F p i a) A"

```

Read semantically: if the hypothesis list is satisfied, the base instance $p[v_i \mapsto 0]$ is satisfied, and the step is satisfied for every value m put into the assignment at index i , then the instance at a is satisfied whenever a evaluates to n . The proof establishes $\Lambda m. m N \Rightarrow \text{sat_fuel } p \text{ (asn_put } A \text{ i m)}$ by QGA induction on m , with the base case supplied by sat_fuel_subst_FD applied to the encoded zero, and then instantiates it at n . The freshness premise $\text{fresh_H } i \text{ } G$ is what allows the assignment to be modified at i without disturbing satisfaction of the hypotheses. $\text{check_ind_sound_fuel}$ then supplies the three premises from the corresponding judgments found in the proof list.

4.4 From steps to proofs

The lifting step is a dedicated induction principle over proof lists.

```

lemma check_list_induct_N:
  assumes pf: "pf N"
  and base: "ΛJ A. A N ⇒ check_list Nil ⇒ J N ⇒ mem J Nil ⇒
    sat_hyp (hyp_of J) A ⇒ sat (conc_of J) A"
  and step: "Λh t. h N ⇒ t N ⇒
    (ΛJ A. A N ⇒ check_list t ⇒ J N ⇒ mem J t ⇒
      sat_hyp (hyp_of J) A ⇒ sat (conc_of J) A) ⇒
    (ΛJ A. A N ⇒ check_list (Cons h t) ⇒
      J N ⇒ mem J (Cons h t) ⇒
      sat_hyp (hyp_of J) A ⇒ sat (conc_of J) A)"
  shows "ΛJ A. A N ⇒ check_list pf ⇒ J N ⇒ mem J pf ⇒
    sat_hyp (hyp_of J) A ⇒ sat (conc_of J) A"

```

It is stated for sat_fuel and sat_hyp_fuel . Its only caller instantiates it at those two, and the indexed predicates it uses internally are defined in terms of them.

Internally it is proved by list_induct on a statement of the form

$$\forall A. \forall K. \forall k. \text{check_list } pf \rightarrow (\text{mem } K \text{ pf} \rightarrow (\text{sat_hyp_at } k \text{ (hyp_of } K) \text{ A} \rightarrow \text{sat_fuel} (\text{conc_of } K) \text{ A})).$$

The predicate being inducted on has to be a single formula, so the generalizations over the assignment, over the judgment and over the budget sit inside it as QGA quantifiers.

The three antecedents are why the budget is explicit. Each one has to be grounded before $\rightarrow$ can be introduced, and check_list_bool , mem_bool and sat_hyp_at_bool supply exactly that at the three levels. The conclusion needs no such treatment, since $\rightarrow$ introduction constrains only the antecedent, which is why sat_fuel appears there unindexed. With sat_hyp_fuel in the antecedent instead, the introduction would be blocked.

The indexed predicate never leaves the proof. The statement of the lemma is in sat_hyp_fuel and sat_fuel , and the bridges of §3.9 convert at the boundary. sat_hyp_atD runs downward at the two points where the induction hands control to base and step, both of which are

stated in the unindexed predicates. `sat_hyp_at_exI` runs upward twice, once in the tail case where the induction hypothesis needs a budget for the hypotheses of another judgment, and once at the end where the lemma's own assumption has to produce one.

Instantiating the principle and discharging the step case with `valid_step_sound_fuel` gives soundness for every judgment occurring in a checked list.

```
lemma check_list_sound_fuel_N:
  assumes pf: "pf N"
  shows "\J A. A N  $\implies$  check_list pf  $\implies$ 
    J N  $\implies$  mem J pf  $\implies$ 
    sat_hyp_fuel (hyp_of J) A  $\implies$ 
    sat_fuel (conc_of J) A"
```

Finally `is_valid_proof` is unfolded. Its two guards are decided with `cases_bool`, the head of the proof is shown to be a member of it by `cons_reconstr` and `mem_cons_head`, and the previous lemma applies.

```
lemma soundness_bridge_fuel:
  assumes vp: "is_valid_proof p J"
    and satG: "sat_hyp_fuel (hyp_of J) A"
    and AN: "A N"
  shows "sat_fuel (conc_of J) A"
```

4.5 Consistency

The two formula constructors are defined, and their satisfaction is unpacked.

```
definition mk_eq :: "num  $\Rightarrow$  num  $\Rightarrow$  num" where
  "mk_eq a b  $\equiv$  pack_F 0 {a, b}"

definition mk_neq :: "num  $\Rightarrow$  num  $\Rightarrow$  num" where
  "mk_neq a b  $\equiv$  pack_F 1 {a, b}"
```

```
lemma sat_fuel_mk_eqE:
  assumes a: "a N" and b: "b N" and sat: "sat_fuel (mk_eq a b) A"
  obtains q where "q N" and "evals a A q" and "evals b A q"
```

```
lemma sat_fuel_mk_neqE:
  assumes a: "a N" and b: "b N" and sat: "sat_fuel (mk_neq a b) A"
  obtains x y where "x N" and "y N" and "evals a A x" and "evals b A y" and "x  $\neq$  y"
```

Both unpacking lemmas go through `tag_pack_F` and `load_pack_F` to recover the two sides from the code, and then through the definition of `sat_fuel`. Groundedness of proof checking, the last assumption of the skeleton, is `proof_is_bool`, obtained from `check_list_bool` by list induction and from a chain of `condTB` steps over the branches of `valid_step`.

With these in place, `bga_fuller`, the union of the step-indexed semantics with the proof checker, is declared a sublocale of consistent.

```
sublocale consistent mk_eq mk_neq dfns is_valid_proof evals sat_fuel sat_hyp_fuel
proof (unfold_locales)
  show "\a b. a N  $\implies$  b N  $\implies$  mk_eq a b N"
  by (rule mk_eq_N')
  show "\a b. a N  $\implies$  b N  $\implies$  mk_neq a b N"
  by (rule mk_neq_N')
```

```

show "∧p J. p N ⇒ J N ⇒ is_valid_proof p J B"
  by (rule proof_is_bool)
show "∧A. sat_hyp_fuel Nil A"
  by (rule sat_hyp_fuel_nil)
show "∧t A r q. t N ⇒ A N ⇒ evals t A r ⇒ evals t A q ⇒ r = q"
  by (rule evals_functional)
show "∧a b A R. a N ⇒ b N ⇒ sat_fuel (mk_eq a b) A ⇒
      (∧q. q N ⇒ evals a A q ⇒ evals b A q ⇒ R) ⇒ R"
  by (rule sat_fuel_mk_eqE)
show "∧a b A R. a N ⇒ b N ⇒ sat_fuel (mk_neq a b) A ⇒
      (∧x y. x N ⇒ y N ⇒ evals a A x ⇒ evals b A y ⇒ x ≠ y ⇒ R) ⇒ R"
  by (rule sat_fuel_mk_neqE)
show "∧p J A. is_valid_proof p J ⇒ sat_hyp_fuel (hyp_of J) A ⇒ A N ⇒ sat_fuel (conc_of
  J) A"
  by (rule soundness_bridge_fuel)

```

Each obligation is discharged by one of the lemmas above: `mk_eq_N'` and `mk_neq_N'` for `habeas quid`, `proof_is_bool` for groundedness of the checker, `sat_hyp_fuel_nil` for the empty hypothesis list, `evals_functional` for determinism, `sat_fuel_mk_eqE` and `sat_fuel_mk_neqE` for the two unpacking assumptions, and `soundness_bridge_fuel` for soundness. The theorem `syntactically_consistent` of §3.4 becomes available inside `bga_fuller`, and it remains to exhibit an encoding that satisfies the interface.

5 The concrete encoding

`encode.thy` instantiates the interface of §3.6 with a Gödel numbering built from the Cantor pairing of §2.11, supplies concrete recursive definitions for every function the checker is built from, discharges all the assumptions, and instantiates the consistency theorem.

5.1 The encoders

A code is a pair whose first component is the constructor `tag` and whose second is the payload. Formulas use the pairing directly, and terms compose it with an involution that exchanges 0 and 1.

```

definition swap01 :: "num ⇒ num" where
  "swap01 t ≡ if t = 0 then 1 else if t = 1 then 0 else t"

definition tag_T :: "tm ⇒ tmtag"          where "tag_T n      ≡ swap01 (cpx n)"
definition load_T :: "tm ⇒ tm"             where "load_T n      ≡ cpy n"
definition pack_T :: "tmtag ⇒ tm ⇒ tm"    where "pack_T tg L ≡ ⟨swap01 tg, L⟩"
definition tag_F :: "fm ⇒ fmtag"          where "tag_F n      ≡ cpx n"
definition load_F :: "fm ⇒ tm"             where "load_F n      ≡ cpy n"
definition pack_F :: "fmtag ⇒ fm ⇒ fm"    where "pack_F tg L ≡ ⟨tg, L⟩"

```

Remark 5.1 (Why `swap01`). The interface requires `tag_T 0 = T_ZERO`, so the code 0 must denote the term 0. With `T_ZERO = 1` and `cpx 0 = 0`, taking `tag_T = cpx` would make the code 0 a variable reference, since `T_VAR = 0`, and the structural-decrease assumption $t \neq 0 \Rightarrow \text{load}_T t < t = 1$ would then have no atomic base to stop at. Composing with `swap01` on both the packing and the unpacking side relabels those two tags and leaves everything else alone, and `swap01 (swap01 t) = t` makes the composition cancel. Formulas need no adjustment, since $F_{EQ} = 0$ already.

```

lemma swap01_0 [simp]: "swap01 0 = 1"
  unfolding swap01_def by (cases bool: "(0::num) = 0", simp+)

lemma swap01_N [simp, auto]: "t N  $\implies$  swap01 t N"
  unfolding swap01_def
  apply (cases bool: "t = 0", simp+)
  done

```

swap01_N and swap01_inv are proved by cases bool: on the guards. The habeas quid premise t N is what makes t = 0 grounded and the case split available.

5.2 Concrete definitions

Every function the locales left abstract is introduced in a single axiomatization block whose axioms are their recursive := equations. This is the definitional mechanism of §2.8 applied at scale, and it is why a self-referential evaluator can be written down at all: no termination obligation is incurred when the definition is made.

```

axiomatization
  eval          :: "tm  $\Rightarrow$  asn  $\Rightarrow$  val"                and
  eval_fuel     :: "num  $\Rightarrow$  tm  $\Rightarrow$  asn  $\Rightarrow$  val" and
  subst_T       :: "tm  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  tm"          and
  subst_F       :: "fm  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  fm"          and
  subst_body    :: "tm  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  tm"          and
  check_template :: "fm  $\Rightarrow$  fm  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  fm  $\Rightarrow$  tm  $\Rightarrow$  o" and
  rep_vars_T    :: "tm  $\Rightarrow$  tm  $\Rightarrow$  tm"                and
  rep_vars_F    :: "fm  $\Rightarrow$  tm  $\Rightarrow$  fm"                and
  find_phi      :: "jdg  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  pf  $\Rightarrow$  o"  and
  find_eq       :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  pf  $\Rightarrow$  o"          and
  check_subst   :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  o"                and
  check_eq_rules :: "hyp  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  tmtag  $\Rightarrow$  tmtag  $\Rightarrow$  pf  $\Rightarrow$  o" and
  check_neq_rules :: "hyp  $\Rightarrow$  tm  $\Rightarrow$  tm  $\Rightarrow$  tmtag  $\Rightarrow$  tmtag  $\Rightarrow$  pf  $\Rightarrow$  o" and
  check_ind_template :: "fm  $\Rightarrow$  fm  $\Rightarrow$  tm  $\Rightarrow$  hyp  $\Rightarrow$  fm  $\Rightarrow$  tm  $\Rightarrow$  pf  $\Rightarrow$  o" and
  find_ind_base  :: "jdg  $\Rightarrow$  tm  $\Rightarrow$  pf  $\Rightarrow$  pf  $\Rightarrow$  o"  and
  check_ind      :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  o"                and
  find_cut       :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  pf  $\Rightarrow$  o"          and
  check_cut      :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  o"                and
  find_struct    :: "jdg  $\Rightarrow$  hyp  $\Rightarrow$  pf  $\Rightarrow$  o"        and
  check_struct   :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  o"                and
  app_try        :: "jdg  $\Rightarrow$  num  $\Rightarrow$  num  $\Rightarrow$  num  $\Rightarrow$  pf  $\Rightarrow$  o" and
  app_y          :: "jdg  $\Rightarrow$  num  $\Rightarrow$  num  $\Rightarrow$  num  $\Rightarrow$  pf  $\Rightarrow$  o" and
  app_x          :: "jdg  $\Rightarrow$  num  $\Rightarrow$  num  $\Rightarrow$  pf  $\Rightarrow$  o"  and
  app_d          :: "jdg  $\Rightarrow$  num  $\Rightarrow$  pf  $\Rightarrow$  o"        and
  check_app      :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  o"                and
  valid_step     :: "jdg  $\Rightarrow$  pf  $\Rightarrow$  o"                and
  check_list     :: "pf  $\Rightarrow$  o"                        and
  is_valid_proof :: "pf  $\Rightarrow$  jdg  $\Rightarrow$  o"                and
  fresh_T        :: "num  $\Rightarrow$  tm  $\Rightarrow$  o"                and
  fresh_F        :: "num  $\Rightarrow$  fm  $\Rightarrow$  o"                and
  fresh_H        :: "num  $\Rightarrow$  hyp  $\Rightarrow$  o"                and
  dfn_is         :: "dfn  $\Rightarrow$  num  $\Rightarrow$  tm  $\Rightarrow$  o"        and
  asn_put        :: "asn  $\Rightarrow$  num  $\Rightarrow$  val  $\Rightarrow$  asn"

```

Thirty-three constants: the two evaluators, the three substitutions, the freshness predicates, the assignment update, and the checker hierarchy down to is_valid_proof. Their equations repeat the locale assumptions verbatim, for instance the direct evaluator

```

eval_def: "eval t A :=
  if tag_T t = T_VAR then nth (load_T t) A
  else if tag_T t = T_ZERO then 0
  else if tag_T t = T_SUC then S(eval (load_T t) A)
  else if tag_T t = T_PRED then P(eval (load_T t) A)
  else if tag_T t = T_IFZ then
    (if eval (cpx (load_T t)) A = 0
     then eval (cpx (cpy (load_T t))) A
     else eval (cpy (cpy (load_T t))) A)
  else
    (if eval (cpx (cpy (load_T t))) A = eval (cpx (cpy (load_T t))) A
     then (if eval (cpy (cpy (load_T t))) A = eval (cpy (cpy (load_T t))) A
            then eval (nth (cpx (load_T t)) dfns)
            ((eval (cpx (cpy (load_T t))) A) > ((eval (cpy (cpy (load_T t))) A) > Nil))
            else 0)
     else 0)" and

```

and the assignment update

```

asn_put_def: "asn_put A i v :=
  if i = 0 then
    if A = Nil then v > Nil
    else v > list_tl A
  else if A = Nil then 0 > asn_put Nil (i - 1) v
  else list_hd A > asn_put (list_tl A) (i - 1) v" and

```

The background definition list stays opaque. Only its termination is assumed:

```

axiomatization dfns :: "dfn" where dfns_N: "dfns N"

```

so the result holds for every terminating definition list, including lists whose definitions diverge on all arguments.

5.3 Discharging the assumptions

What remains is the arithmetic that the locale structure confined to this one file.

Habeas quid. Each of the six encoder operations preserves termination, by the corresponding pairing lemma.

```

lemma pack_F_N:
  assumes t: "t N" and L: "L N" shows "pack_F t L N"
  unfolding pack_F_def by (rule cpair_terminates[OF t L])

lemma tag_F_N:
  assumes f: "f N" shows "tag_F f N"
  unfolding tag_F_def by (rule cpx_terminates[OF f])

lemma load_F_N:
  assumes f: "f N" shows "load_F f N"
  unfolding load_F_def by (rule cpy_terminates[OF f])

lemma pack_T_N:
  assumes t: "t N" and L: "L N" shows "pack_T t L N"
  unfolding pack_T_def by (rule cpair_terminates[OF swap01_N[OF t] L])

lemma tag_T_N:
  assumes t: "t N" shows "tag_T t N"
  unfolding tag_T_def by (rule swap01_N[OF cpx_terminates[OF t]])

```



```

lemma load_T_N:
  assumes t: "t N" shows "load_T t N"
  unfolding load_T_def by (rule cpy_terminates[OF t])

```

Round-trips. Injectivity is the projection equations of §2.11 composed with `swap01_inv`. The retraction property is `cpair_reonstr`, that a number is the pair of its projections.

```

lemma tag_pack_F:
  assumes t: "t N" and L: "L N" shows "tag_F (pack_F t L) = t"
  unfolding tag_F_def pack_F_def by (rule cpx_proj[OF t L])

lemma load_pack_F:
  assumes t: "t N" and L: "L N" shows "load_F (pack_F t L) = L"
  unfolding load_F_def pack_F_def by (rule cpy_proj[OF t L])

lemma tag_pack_T:
  assumes t: "t N" and L: "L N" shows "tag_T (pack_T t L) = t"
  unfolding tag_T_def pack_T_def
  using t by (simp add: cpx_proj[OF swap01_N[OF t] L] swap01_inv[OF t])

lemma load_pack_T:
  assumes t: "t N" and L: "L N" shows "load_T (pack_T t L) = L"
  unfolding load_T_def pack_T_def by (rule cpy_proj[OF swap01_N[OF t] L])

lemma pack_tag_F:

```

Structural decrease. Both payload extractors are `cpy`, so the assumption reduces to strict decrease of the second projection on nonzero arguments, which is already available in `GD.thy`. The base case is `swap01 (cpx 0) = swap01 0 = 1`.

```

lemma decrease_F:
  assumes f: "f N" and nz: "f ≠ 0" shows "load_F f < f = 1"
  unfolding load_F_def using f nz by simp

lemma decrease_T:
  assumes t: "t N" and nz: "t ≠ 0" shows "load_T t < t = 1"
  unfolding load_T_def using t nz by simp

lemma tag_T_zero: "tag_T 0 = T_ZERO"
  unfolding tag_T_def by simp

```

Monotonicity. This is the assumption that needs work. The chain begins with `pair_step`, that a single successor step in the second component does not decrease the pair,

$$\text{pair_step: } \llbracket a \text{ N}; k \text{ N} \rrbracket \implies \langle a, k \rangle \leq \langle a, S \ k \rangle = 1,$$

which follows from `cpair_suc` and repeated use of `leq_monotone_add_r`. `pair_mono_2` closes it under induction on the second component, and the two interface obligations follow.

```

lemma mono_pack_T:
  assumes tg: "tg N" and x: "x N" and y: "y N" and h: "x ≤ y = 1"
  shows "pack_T tg x ≤ pack_T tg y = 1"
  unfolding pack_T_def by (rule pair_mono_2[OF swap01_N[OF tg] x y h])

lemma mono_pack_F:
  assumes tg: "tg N" and x: "x N" and y: "y N" and h: "x ≤ y = 1"
  shows "pack_F tg x ≤ pack_F tg y = 1"

```

```
unfolding pack_F_def by (rule pair_mono_2[OF tg x y h])
```

The induction in `pair_mono_2` is a good miniature of the style of the whole development. The statement being inducted on is the object-level implication $(x \leq y = 1) \rightarrow ((a, x) \leq (a, y) = 1)$, every `implI` step first discharges a groundedness obligation through `eqBool`, and the case analysis in the step case goes through `cases_bool`.

5.4 Instantiation and the theorem

With the assumptions available the locale is interpreted. The interpretation supplies, in order, the six encoder operations, the three freshness predicates, the definition list and its membership test, the step-indexed evaluator, the three substitutions and the assignment update, and the checker hierarchy.

```
interpretation conc2: bga_fuller
  tag_T load_T pack_T tag_F load_F pack_F fresh_T fresh_F fresh_H dfns dfn_is
  eval_fuel
  subst_T subst_F subst_body asn_put
  check_template rep_vars_T rep_vars_F find_phi find_eq check_subst
  check_eq_rules check_neq_rules
  check_ind_template find_ind_base check_ind
  find_cut check_cut find_struct check_struct
  app_try app_y app_x app_d check_app
  valid_step check_list is_valid_proof
  apply unfold_locales
  apply (fact dfns_N pack_F_N tag_F_N load_F_N pack_T_N tag_T_N load_T_N
    tag_pack_F load_pack_F tag_pack_T load_pack_T pack_tag_F pack_tag_T
    decrease_F decrease_T tag_T_zero mono_pack_T mono_pack_F
    fresh_T_def fresh_F_def fresh_H_def dfn_is_def eval_fuel_def
    subst_T_def subst_F_def subst_body_def asn_put_def
    check_template_def rep_vars_T_def rep_vars_F_def find_phi_def
    find_eq_def check_subst_def check_eq_rules_def check_neq_rules_def
    check_ind_template_def find_ind_base_def check_ind_def
    find_cut_def check_cut_def find_struct_def check_struct_def
    app_try_def app_y_def app_x_def app_d_def check_app_def
    valid_step_def check_list_def is_valid_proof_def)+
done
```

Every goal generated by `unfold_locales` is closed by `fact`: the structural obligations by the lemmas above, and the definitional obligations by the `:=` axioms of the axiomatization block, which are literally the equations the locale assumes.

```
theorem BGA_syntactically_consistent:
  assumes "a N" and "b N" and "p1 N" and "p2 N"
  shows "¬ (is_valid_proof p1 (Nil, conc2.mk_eq a b) ∧
    is_valid_proof p2 (Nil, conc2.mk_neq a b))"
  using assms by (rule conc2.syntactically_consistent)
```

Read out, the theorem says that for all terminating a , b , p_1 and p_2 , it is not the case that p_1 codes a BGA proof of ' $a = b$ ' from no hypotheses while p_2 codes a BGA proof of ' $a \neq b$ ' from no hypotheses. Every symbol in that statement, the quantifiers, the negation, the conjunction and the proof checker, belongs to QGA.

6 Reduction of QGA to BGA

§4 proves BGA consistent inside QGA. To turn that into a self-consistency result, QGA itself has to be reduced to BGA, so that the system doing the proving is no stronger than the system being proved consistent. This section is to be written.

7 A big-step semantics for QGA

§6 is one route to a self-consistency result. This section sketches the other, which is to give QGA a semantics of its own and repeat the argument of §4 with QGA in place of BGA. Nothing here is formalized yet. The purpose is to fix the shape of the semantics before the encoding work begins, because one clause in it determines the architecture of everything that follows.

7.1 What the proof needs

The consistency argument of §4 has three moving parts. Evaluation is deterministic, the proof checker is sound with respect to evaluation, and False evaluates to false. Consistency is then immediate: a proof of False would make False evaluate to true, and determinism forbids that.

Two constraints follow, and they are the constraints that matter. The semantics has to be expressible as a QGA recursive definition, since it will live inside QGA rather than above it. And the satisfaction predicate has to be decided at each fixed index, because the soundness proof is an induction whose motive carries satisfaction of the hypotheses in an antecedent position, and $\rightarrow$ introduction demands that the antecedent be grounded. This second constraint is the whole reason §3.8 indexes evaluation by fuel, and it is the constraint that decides the design below.

7.2 Terms

Term evaluation is the semantics of §3.8 unchanged. A judgment $\langle t, A \rangle \Downarrow_k v$ says that the term coded by t , under the assignment A , evaluates to the value v within k steps. It is realized as a QGA function `eval_fuel` returning $S(v)$ on success and 0 on timeout, which makes it total, so equality of results is decidable and the groundedness obligations discharge themselves. Divergence is the absence of any k , and the relation $\langle t, A \rangle \Downarrow v$ abbreviates $\exists k. \langle t, A \rangle \Downarrow_k v$. Fuel is monotone, so a successful evaluation stays successful when more is supplied.

The definition list is a parameter of the semantics. A constant introduced by $f \bar{x} := \text{body}$ is evaluated by looking its body up and continuing, exactly as `dfns` is consulted in §3.

7.3 Formulas

Formula evaluation is two-valued with gaps. A formula either evaluates to `tt`, or to `ff`, or to neither, and the third case is what groundedness means. Writing $\langle \varphi, A \rangle \Downarrow \text{tt}$ and $\langle \varphi, A \rangle \Downarrow \text{ff}$ for the two judgments, the clauses for the primitives are the strong three-valued ones.

$\langle s = t, A \rangle \Downarrow \text{tt}$	if $\langle s, A \rangle \Downarrow n$ and $\langle t, A \rangle \Downarrow n$
$\langle s = t, A \rangle \Downarrow \text{ff}$	if $\langle s, A \rangle \Downarrow m, \langle t, A \rangle \Downarrow n$ and $m \neq n$
$\langle \neg\varphi, A \rangle \Downarrow \text{tt}$	if $\langle \varphi, A \rangle \Downarrow \text{ff}$
$\langle \neg\varphi, A \rangle \Downarrow \text{ff}$	if $\langle \varphi, A \rangle \Downarrow \text{tt}$
$\langle \varphi \vee \psi, A \rangle \Downarrow \text{tt}$	if $\langle \varphi, A \rangle \Downarrow \text{tt}$, or if $\langle \psi, A \rangle \Downarrow \text{tt}$
$\langle \varphi \vee \psi, A \rangle \Downarrow \text{ff}$	if $\langle \varphi, A \rangle \Downarrow \text{ff}$ and $\langle \psi, A \rangle \Downarrow \text{ff}$
$\langle \text{if } c \text{ then } \varphi \text{ else } \psi, A \rangle \Downarrow v$	if $\langle c, A \rangle \Downarrow \text{tt}$ and $\langle \varphi, A \rangle \Downarrow v$
$\langle \text{if } c \text{ then } \varphi \text{ else } \psi, A \rangle \Downarrow v$	if $\langle c, A \rangle \Downarrow \text{ff}$ and $\langle \psi, A \rangle \Downarrow v$

Each clause is positive, so the whole definition is a monotone induction. An equation is true when both sides converge to the same value and false when they converge to different ones, which is where *habeas quid* comes from: a divergent side gives neither clause, so the equation is ungrounded, and a $\mathbb{N}$ is grounded exactly when a converges. Disjunction is true as soon as one disjunct is, whatever the other does, which is what makes $\vee\text{I1}$ and $\vee\text{I2}$ sound while excluded middle is not. Nothing evaluates to tt and ff together, which is determinism and is proved by induction on the two derivations.

7.4 The quantifier clause

Six of the eight clauses above are bounded searches, so they are Σ_1 and a QGA recursive definition can perform them. The quantifiers are where a choice has to be made, and the two candidate readings differ in exactly the property that §7 identified as decisive.

The first reading is instance-wise.

$$\begin{aligned} \langle \forall x. \varphi, A \rangle \Downarrow \text{tt} & \quad \text{if } \langle \varphi, A[x \mapsto n] \rangle \Downarrow \text{tt} \text{ for every } n \\ \langle \forall x. \varphi, A \rangle \Downarrow \text{ff} & \quad \text{if } \langle \varphi, A[x \mapsto n] \rangle \Downarrow \text{ff} \text{ for some } n \end{aligned}$$

This is the reading a classical reader expects, and it makes soundness of $\forall\text{I}$ a substitution lemma and nothing more. QGA is first order, its terms denote numbers or nothing, and it has no term-level binder, so assignments suffice and no strengthening of the induction motive is required. Determinism at the quantifier is equally easy, since one instance cannot be both true and false.

Its defect is that the true clause quantifies over all n . The predicate is not Σ_1 , so no QGA definition decides it, and the internal soundness proof would have satisfaction sitting undecided in an antecedent. That is precisely the obstruction §4 had to work around for BGA, and there the obstruction was removable because the undecidability came from an unbounded search for fuel. Here it comes from a universal quantification over the numerals, and fuel does not touch it.

The second reading grounds a universal in the system's own proof search.

$$\begin{aligned} \langle \forall x. \varphi, A \rangle \Downarrow \text{tt} & \quad \text{if some valid QGA proof derives } x \mathbb{N} \vdash \varphi \\ \langle \forall x. \varphi, A \rangle \Downarrow \text{ff} & \quad \text{if some valid QGA proof derives } \vdash \neg\varphi[x \mapsto \bar{n}] \text{ for some numeral } \bar{n} \end{aligned}$$

Both clauses are searches over proof codes, so the predicate stays Σ_1 and a fuelled version of it is total and decided at each index, which is what the soundness induction needs. Both mention derivability positively, so the simultaneous definition of the checker and the semantics remains a monotone induction and no Tarskian circularity arises. The clauses for $\exists$ are the duals, with the true clause a search for a witness and the false clause a search for a proof of the schematic negation.

This reading also settles the two axioms of §2.6. $\neg\forall\text{I}$ is the false clause read from right to left, since a derivation of $\neg(Qa)$ together with $a \mathbb{N}$ is the certificate the clause asks for, and $\neg\forall\text{E}$

is the same clause read from left to right. Under the instance-wise reading they are equally sound. Under either one they are justified rather than postulated, which is the reason they were added.

We propose the second reading. The cost is the architecture it forces. The soundness case for $\forall I$ has to produce a certificate with no residual hypotheses, while the induction hypothesis supplies only that the premise judgment is semantically true, so the hypotheses have to be converted back into derivations and discharged by cut . That step is an internal completeness lemma, and it has to be established before soundness rather than after. Reflective grounded arithmetic is organized this way for the same reason, and it is the main risk in the plan.

7.5 Obligations

Assuming the second reading, a self-consistency proof for QGA needs the following, in this order.

1. An arithmetization of QGA's own syntax, judgments, proofs and proof checker, following §3. This is needed to state consistency at all, and the reflective clauses need it as well.
2. The fuelled evaluation of terms and formulas, with monotonicity in the index, and the decidedness lemma for satisfaction at a fixed index.
3. Internal completeness: a formula that evaluates to tt is derivable.
4. Soundness: every rule of §2 preserves truth. The propositional cases read off the clauses directly, the equality cases need a congruence lemma for evaluation, IND needs induction on the value of its subject, and DEFE and DEFI need that a definition and its body evaluate alike.
5. Determinism, which under the reflective reading rests on soundness, because a colliding pair of certificates has to be instantiated at a common numeral before the contradiction appears.
6. $\langle \text{False}, A \rangle \Downarrow \text{ff}$, which is one evaluation.

Consistency follows from the last three. Two of these items deserve a warning. Item 3 is the one §7.4 identified as the cost of the design. Item 4 has a trap of its own at the substitution rule: the obvious motive, truth under all satisfying assignments, is not inductive when substitution can replace a variable by a term that changes what the hypotheses say. For a first-order system the standard motive is expected to suffice, and confirming that is part of the work.

7.6 What the semantics does not settle

Grounded truth under either reading is only recursively enumerable, so the diagonal argument produces an ungrounded sentence rather than a true and unprovable one, and a system that is consistent, effective and complete for its own semantics is not excluded. That is why the arrangement is available at all.

What remains open is whether the resulting proof deserves the name self-consistency. The semantics interprets QGA's $\forall$ by a search over QGA proofs, so the argument is carried out in the same system it is about, and a Kripkean partial truth predicate is what makes that admissible. A sceptical reader is entitled to ask for the adequacy statement relating the arithmetized syntax of item 1 to the Pure-level formulas of §1.3, and the present development does not supply it. §6 avoids the question by a different route.

8 QGA on QGA: feasibility and the development skeleton

§7 proposed a semantics for QGA and listed six obligations for a self-consistency proof, flagging item 3, internal completeness, as the main risk. This section reports on that plan. The verdict is that the result is reachable, that the plan as stated does not close, and that a change to one clause of the semantics removes the obstruction that made it not close. The theories `pure/QGA_on_GA.thy` and `pure/qga_encode.thy` implement the revised plan: the locale tower, every definition, the abstract consistency skeleton with its theorem proved in full, and the statement of every remaining lemma. They are in the GD session and check.

8.1 Is it possible?

Nothing in Gödel’s second theorem forbids it. Both derivability conditions that carry the proof fail here. QGA is not classical, so the diagonal sentence is ungrounded rather than true-and-unprovable, and grounded truth is recursively enumerable, which is what lets an effective system be complete for its own semantics. More sharply, the internal necessitation and distribution steps that Löb’s theorem needs are unavailable, because $\rightarrow$ introduction carries a *habeas quid* premise: from a derivation of A one does not get $\text{Prov}(A) \rightarrow \text{Prov}(B)$ for free, since the implication cannot be formed until its antecedent is known to be grounded. Reflective grounded arithmetic proves the corresponding facts — soundness, completeness, consistency — for a system strictly stronger than QGA, so the mathematics is known to work out. What is new here is only the metatheory in which it is carried out.

Nor is the *internal* carrying-out obviously blocked. `BGA_on_GA` already shows that QGA can host a full metamathematical argument about a formal system with no classical ambient logic. The question is therefore not whether grounded arithmetic can do metamathematics at all, but whether the particular obligations that quantifiers add can be met inside it.

8.2 Where the plan of §7 does not close

The architecture of §4 imposes one hard constraint, and it is worth restating because it drives everything. The soundness induction runs over a proof list, QGA induction instantiates a schematic predicate, so the generalizations over the assignment, the judgment and the fuel must live inside the object-level formula being inducted on, and the hypothesis-satisfaction premise therefore sits in the antecedent of an object-level $\rightarrow$. Introducing that implication requires the antecedent to be grounded. So:

satisfaction of a hypothesis list, at a fixed index, must be decided.

That rules out the instance-wise reading of $\forall$, as §7.4 says, and forces the reflective one. So far the report is right.

The difficulty appears one step later. Under the reflective reading, three facts that a classical development proves independently become entangled, and they are entangled in a cycle:

- Soundness of $\forall E$ needs the certificate found by the semantics to be sound, that is, it needs soundness.
- Soundness of $\exists x F$ needs that no formula is both true and false, that is, it needs determinism.
- Determinism at a universal needs to rule out a truth certificate colliding with a falsity witness, which again needs soundness.

Reflective grounded arithmetic hits the same cycle and breaks it in two steps: first internal completeness, proved independently, then a *witnessed* variant of the evaluation relation whose quantifier clauses carry, as extra positive premises, the semantic facts their certificates promise. Determinism and the $\forall E$ case are immediate at the witnessed level, and the two semantics are reconciled afterwards.

Transplanting that directly fails. The witnessed clause for $\forall$ carries “every numeral instance is true”, which is not decided at any fixed index, so a witnessed satisfaction predicate cannot be the one sitting in the antecedent.

8.3 The two changes that make it close

Two changes to the semantics of §7, both small, remove the cycle. Neither weakens the encoded system.

Split the semantics in two, and use each where it belongs. Decidedness is needed only for the *hypotheses*; the conclusion of the soundness statement may be as undecided as one likes, since $\rightarrow$ introduction constrains only the antecedent. So the development carries two layers.

	official	witnessed
symbols	evtt k f A, evff k f A	wsat f A, wunsat f A
kind	total QGA functions	QGA recursive predicates
decided?	yes, at every fixed k	no, and it does not matter
occurs in	antecedents	conclusions

The witnessed clauses are the official ones with the certificate’s semantic content added as a conjunct:

$$\text{wsat } (\forall v_i. \varphi) A := (\exists k. \text{evtt } k \text{ } (\forall v_i. \varphi) A = 1) \wedge (\forall n. \text{wsat } \varphi (\text{asn_put } A \text{ } i \text{ } (S \text{ } n)))$$

Exclusivity, $\neg(\text{wsat } f A \wedge \text{wunsat } f A)$, then becomes a plain strong induction on the formula code: a universal that is both carries truth at every instance and falsity at one, so the collision reappears at a strictly smaller code. No appeal to soundness, so no cycle. Soundness of exF is then available, and soundness of $\forall E$ reads its instance straight off the clause instead of re-proving the certificate sound.

Keep truth and falsity as two separate searches. The obvious design makes formula evaluation a single three-valued function. That forces a priority between the two quantifier searches, and whichever loses can be preempted at a larger index, so monotonicity fails; recovering it needs exclusivity, which needs soundness. Two independent monotone searches *evtt* and *evff* have no tie to break. Exclusivity is simply not claimed of them — it is a theorem about the witnessed layer — and the two bridges $\text{wsat } f A \Rightarrow \text{sat } f A$ and $\text{wunsat } f A \Rightarrow \text{unsat } f A$ then hold without either having to rule the other out.

8.4 The clause that removes the completeness obligation

§7.4 takes the certificate for $\forall x. \varphi$ to be a proof of $x N \vdash \varphi$ with no residual hypotheses, and notes that the $\forall I$ case must then convert satisfied hypotheses back into derivations and discharge them by *cut*, which is internal completeness, “the main risk in the plan”. That obligation is an artefact of the clause, not of the design. The development uses instead a *context-relative* certificate. A witness $w = \langle j, \langle q, \Gamma \rangle \rangle$ is a tt-witness for $\forall v_i. \varphi$ under A when

$$q \text{ is a valid proof of } \Gamma, v_i N \vdash \varphi[A], \quad \Gamma \text{ is closed,} \quad \Gamma \text{ is satisfied by } A \text{ within } j \text{ steps,}$$

where $\varphi[A]$ is the closure of φ against A sparing v_i . Read semantically this is “the system proves the schematic instance from premises that are themselves true here” — the original clause composed with the cut that internal completeness was there to perform. Its effect on the proof is that $\forall I$ no longer has to *construct* a certificate: the certificate is the suffix of the proof list the checker is already looking at, instantiated by the closing substitution, which is one application of the `INST` rule.

Requiring Γ to be closed is what keeps the clause stable under substitution: a closed context’s satisfaction does not depend on the assignment, so the substitution lemma never has to move it. Closing φ against A is what keeps the two sides of the substitution lemma comparable; they differ only in that one carries the closed term $a[A]$ where the other carries the numeral of its value, and one `EQSUBST` step converts between them once $\vdash a[A] = \bar{v}$ is available.

The net effect on the obligation list of §7 is:

§7 obligation	status
1. arithmetization of QGA’s syntax and checker	unchanged, and the bulk of the work
2. fuelled evaluation, monotone, decided at each index	unchanged
3. internal completeness	<i>removed</i> , except at terms (see below)
4. soundness of every rule	unchanged
5. determinism	<i>now independent of soundness</i>
6. False is false	unchanged, one evaluation

Item 3 does not vanish entirely. What survives is *term* value completeness, $t \Downarrow v \Rightarrow \vdash t = \bar{v}$, which is needed only to convert a certificate along the substitution lemma. That is a self-contained induction on the fuel inside the BGA fragment, with one proof list per term constructor. It is a fraction of the formula-level completeness the original plan required, and it does not interact with soundness.

8.5 What the development contains

Theory	Imports	Contributes
QGA_on_GA.thy	GD	QGA arithmetized in QGA: syntax, checker, both semantic layers, the soundness statement
qga_encode.thy	QGA_on_GA	one concrete encoding, the interpretation, the theorem

The locale tower mirrors `BGA_on_GA.thy`: `qga_suff_syntax` $\subset$ `qga_suff_semantics` $\subset$ `qga_consistent` for the abstract skeleton, then `qga_encoding` $\subset$ `qga_subst` $\subset$ `qga_dfns`, then one locale per rule group (`qga_struct_rule`, `qga_prop_rule`, `qga_eq_rule`, `qga_cond_rule`, `qga_quant_rule`, `qga_ind_rule`, `qga_app_rule`), merged into `qga_proof_check`, then `qga_fuel_semantics` $\subset$ `qga_witnessed` $\subset$ `qga_full`, which is declared a sublocale of `qga_consistent`.

The encoded syntax is exactly the primitive vocabulary of `GD.thy`. Terms are variables, 0 , $S(\cdot)$, $P(\cdot)$, the conditional and definition application; the conditional’s guard is a *formula*, so term and formula codes are mutually recursive, which is the one structural difference from BGA. Formulas are $=$, $\neg$, $\vee$, $\forall$, $\exists$ and the formula-level conditional; $\wedge$, $\longrightarrow$, $\longleftrightarrow$, $\neq$, N , B , `True` and `False` are abbreviations here as they are in `GD.thy`.

The Pure operators are translated as §1.3 reads them. A rule $\llbracket P; Q \rrbracket \Rightarrow R$ becomes “from $\Gamma \vdash P$ and $\Gamma \vdash Q$ infer $\Gamma \vdash R$ ”. A premise $P \Rightarrow R$, as in $\forall E1$, becomes $P, \Gamma \vdash R$: the hypothesis is discharged. A premise $\wedge x. x \ N \Rightarrow Q \ x$, as in $\forall I$ and `IND`, becomes $v_i \ N, \Gamma \vdash Q[v_i]$ *together with*

the *eigenvariable condition* that v_i occurs neither in Γ nor in the conclusion, discharged by `fresh_H i (hyp_of J)` and `fresh_F i (conc_of J)` with i taken to be $J + 1$, since every variable index occurring in a judgment is below the judgment’s own code. This is BGA’s device in `check_ind`, reused. The four operations Pure supplies for free — `assume`, $\Rightarrow$ introduction and elimination, and instantiation of a schematic variable — become the four explicit rules `HYP`, `WEAK`, `CUT` and `INST`, since nothing supplies them to the encoded system.

One point about `INST` is worth recording, because it fixes the shape of assignments. Pure instantiates a schematic variable with an *arbitrary* term and incurs no *habeas quid* obligation. That is sound only if a free variable of the encoded system may denote nothing, so an assignment maps an index either to $S(v)$, meaning the value v , or to 0, meaning bottom. Quantified variables, by contrast, range over values only, which is exactly what the $x N$ premises of $\forall I$, $\forall E$ and $\exists I$ enforce. This is the variable-range invariant of reflective grounded arithmetic, and adopting it is what makes `INST` unconditional and the substitution lemma free of side conditions.

Every rule is effective without numeric guessing, by one convention: when a premise is determined by the conclusion the checker looks it up with `mem`, and when a premise is larger than the conclusion the checker scans the proof list and decomposes what it finds. Only `INST` and the two template rules — `EQSUBST` and the lazy conditional pair — need a bounded countdown, each bounded by the code of the judgment being justified, as `check_template` is in §3.

8.6 Status and what remains

`qga_consistent.syntactically_consistent` is proved in full: no formula is both provable and refutable from no hypotheses. Because QGA has negation as a primitive, this is the ordinary statement of consistency, and not the matched pair $a = b / a \neq b$ that BGA’s lack of negation forced in §4.5. `qga_encode.thy` discharges every structural assumption of the interface by `unfold_locales` and `fact`, and derives `QGA_syntactically_consistent`.

What is not yet proved is the arithmetic in between. Forty-five lemmas in `QGA_on_GA.thy` and one in `qga_encode.thy` stand as sorry, each with a comment naming the argument that discharges it. They fall into four groups, in increasing order of cost.

1. **Groundedness and totality** (about twenty lemmas). Every encoded function is total and every checker is boolean. These are `condTB` chains over decided guards under `strong_induction` or `list_induct`, and are the direct analogues of `fresh_T_bool`, `eval_fuel_N` and `proof_is_bool`. Mechanical, and the largest group by line count.
2. **Monotonicity and freshness**. `evT_mono`, `evtt_mono`, `evff_mono` and the three `_fresh` lemmas. BGA’s `eval_fuel_add` argument, plus the observation that a witness carrying its own fuel makes the quantifier clauses monotone by construction.
3. **The substitution lemmas** for `wsat` and `wunsat`, and the two internal derivability lemmas `term_value_complete` and `closure_by_inst`. These are the ones that needed the design work above, and they are the only place where the development constructs a proof code rather than consuming one.
4. **Soundness**, one case per rule. The propositional and conditional cases read off the clauses, the equality cases go through the template lemma, `IND` is a semantic induction, and the three interesting cases are the three discussed above.

A realistic estimate, calibrated against `BGA_on_GA.thy` at twenty-three thousand lines for six term constructors, two formula constructors and thirteen rules, is that the finished `QGA_on_GA.thy` is three to five times that. The scaling is in the rule count and the connective count, not in the difficulty: after the two changes above, no step of the argument is harder than its BGA counterpart, and one step, determinism, is easier.

8.7 What this would and would not settle

It would settle that QGA proves an arithmetized statement of its own consistency. The scepticism §7 anticipates remains legitimate and is worth stating plainly: the development supplies no adequacy statement relating the arithmetized syntax to the Pure-level formulas of §1.3, so what is proved consistent is the encoded system, and the claim that the encoded system *is* QGA rests on inspection of the tag tables and the rule list rather than on a theorem. The first-order encoding of binding also makes the encoded system a shade different from the higher-order abstract syntax of `GD.thy`, in the direction §1.3 already flagged. §6 avoids both questions by a different route and remains worth pursuing independently.

9 Status

The three BGA theories are complete in the sense that `encode.thy` discharges every assumption of `bga_fuller` by `unfold_locales` and derives `BGA_syntactically_consistent` from `conc2.syntactically_consistent`. What remains for this document is the detail of the soundness proofs summarized in §4, and the content of §6.

Two further theories, `QGA_on_GA.thy` and `qga_encode.thy`, carry the QGA-on-QGA development of §8. They are in the same session and check, but they are a skeleton: the locale tower, every definition, the abstract consistency skeleton with its theorem proved in full, and the statement of every remaining lemma, of which forty-six stand as sorry. `qga_encode.thy` is generated in part from the locale interface by `pure/gen/gen_qga_encode.py`, so that the recursive equations of the two files cannot drift apart.

Three points about the development itself are worth recording.

1. The concrete functions of `encode.thy`, the evaluator, the substitutions and the checkers, are introduced by an axiomatization block whose axioms are their recursive `:=` equations. This is the native definitional mechanism of GA (§2.8), which is sound because GA places no termination requirement on a recursive definition. It does mean the functions are postulated to satisfy their equations rather than built by a conservative definitional package.
2. The exact evaluator `eval` of `bga_semantics` is retained in the sources but is not on the critical path. The consistency argument runs through the step-indexed `eval_fuel` and the relation `evals` derived from it (§3.8), for which the determinism lemma is available.
3. An earlier version of the development, the locale `bga_full` and its interpretation, is retained inside a comment block in `BGA_on_GA.thy` and is superseded by `bga_fuller`. Nothing described here depends on it.